Министерство образования Московской области

Государственное бюджетное профессиональное образовательное учреждение
Московской области
«Сергиево-Посадский колледж»

Специальность 09.02.07

Информационные системы и программирование

КУРСОВОЙ ПРОЕКТ

Тема: Разработка серверной части информационной системы интерактивного конфигуратора системного блока с проверкой совместимости компонентов для магазина по продаже комплектующих

Студент: Дёрев Никита Андреевич

Руководитель проекта: Каревский Максим Андреевич

«__» марта 2026 г. Подпись _____

Работа защищена с оценкой: _____

г. Сергиев Посад

2026 г.

Содержание

Введение 4

1. Основная часть 6

1.1 Цель и задачи разработки 6

1.2 Общая информация о организации 7

1.3 Организационная структура 8

1.4 Анализ реализуемых на предприятии бизнес-процессов и их связь с разрабатываемой серверной частью интернет-магазина 9

1.5 Перспективы развития серверной части 14

1.6 Анализ серверной архитектуры конкурентов 15

2. Технологическая часть 17

2.1 Постановка задачи 17

2.2 Требования к программе 17

2.3 Разработка диаграмм 22

2.4 Разработка базы данных 28

2.5 Реализация пользовательского интерфейса на стороне сервера 38

2.6 Реализация функциональной системы 39

2.7 Тестирование системы 47

2.8 Руководство пользователя 48

Заключение 61

Библиография 62

Приложение А 64

Приложение В 76

Введение

Продажа компьютерных комплектующих через интернет требует не только удобного внешнего интерфейса, но и устойчивой серверной платформы, которая отвечает за хранение данных, проверку пользовательских действий, выполнение бизнес-правил и защиту учётных записей. Для конфигуратора системного блока это особенно важно, поскольку система должна не просто показывать каталог, а помогать пользователю собирать совместимую конфигурацию, сохранять её, публиковать и использовать повторно.

У магазина «Бит» уже существовал клиентский прототип, собранный на HTML, CSS и JavaScript. Он позволял продемонстрировать внешний вид сайта и базовую механику выбора компонентов, однако оставался ограниченным по возможностям. Данные брались из JSON-файлов, состояние пользователя хранилось на клиенте, а большая часть логики не была привязана к централизованной базе данных. Такой подход подходит для макета, но не для рабочего веб-приложения, рассчитанного на реальных пользователей, разные роли доступа и постоянное обновление каталога.

В данной работе рассматривается создание серверной части информационной системы конфигуратора ПК для магазина комплектующих. Разрабатываемая часть проекта должна связать пользовательский интерфейс с базой данных, перенести ключевую логику на сервер, обеспечить безопасную авторизацию, поддержку шаблонов и публикаций сборок, а также дать администраторам инструменты управления пользователями и каталогом.

Для достижения этой цели в проекте решаются следующие задачи:

Изучить предметную область и определить набор функций, необходимых серверной части;

Спроектировать базу данных MySQL для хранения пользователей, товаров, характеристик, шаблонов, сборок и связанных сущностей;

Реализовать серверное приложение на Laravel с разделением на модели, контроллеры, маршруты и Blade-шаблоны;

настроить регистрацию, вход в систему, управление сессиями и

разграничение доступа по ролям;

организовать API для работы конфигуратора, каталога компонентов, шаблонов, корзины и каталога опубликованных сборок;

обеспечить защиту данных за счёт серверной валидации, механизмов Laravel и безопасного хранения паролей;

реализовать административные функции для управления ролями пользователей и изменением содержимого каталога;

проверить корректность работы системы при типовых сценариях использования.

Итогом выполнения проекта должна стать серверная часть, которая превращает статический макет конфигуратора в полнофункциональную веб-систему с централизованной обработкой данных, расширяемой архитектурой и устойчивой моделью безопасности.

1. Основная часть

1.1 Цель и задачи разработки

Назначение данной работы состоит в создании серверного ядра для конфигуратора системного блока магазина «Бит». Это ядро должно брать на себя хранение и обработку данных о комплектующих, выполнение правил совместимости, авторизацию пользователей, работу с шаблонами и сборками, а также подготовку динамических страниц для браузера. Иными словами, необходимо перевести проект из состояния демонстрационного макета в состояние прикладной информационной системы.

Для выполнения поставленной цели требуется решить ряд связанных задач:

необходимо разработать структуру данных в MySQL. В ней должны быть предусмотрены сущности для товаров, категорий компонентов, типовых технических характеристик, коннекторов, слотов, изображений, пользователей, шаблонов, рабочих сборок, корзин и опубликованных конфигураций.

требуется построить серверную архитектуру на базе Laravel. Для этого создаются модели Eloquent, настраиваются связи между таблицами, пишутся контроллеры обработки запросов и формируется система маршрутов как для HTML-страниц, так и для API, к которому обращается клиентская часть конфигуратора.

нужно реализовать подсистему безопасности. Она включает регистрацию пользователей, вход в систему, хеширование паролей, управление сессией, проверку прав доступа и защиту маршрутов через middleware. Особенно важно обеспечить разделение прав между обычным пользователем, привилегированными ролями и администратором.

необходимо реализовать прикладные функции самого конфигуратора: выдачу каталога компонентов с фильтрами и сортировкой, просмотр детальной карточки товара, сохранение шаблонов, экспорт сборок, добавление сборки в корзину и публикацию готовых конфигураций в общем каталоге.

требуется подготовить административный контур системы, через который можно управлять пользователями, ролями, товарами, изображениями и отдельными элементами контента.

Дополнительно необходимо подготовить проектную документацию, описать структуру базы данных и логику серверной части, а затем провести тестирование ключевых пользовательских сценариев и административных операций.

1.2 Общая информация о организации

Магазин «Бит» представляет собой розничное предприятие, работающее в Сергиевом Посаде Московской области в формате индивидуального предпринимательства. Основная специализация магазина связана с реализацией компьютерной техники и сопутствующих товаров. К направлениям его деятельности относятся:

розничную торговлю компьютерами, комплектующими и периферийными устройствами;

реализацию оргтехники, сетевого оборудования и расходных материалов;

обслуживание и ремонт компьютерной техники в собственном сервисном центре;

консультационную поддержку клиентов по подбору оборудования.

Перечисленные виды деятельности формируют предметную область разрабатываемой системы. В рамках курсового проекта автоматизируется не вся работа магазина, а только тот её сегмент, который связан с онлайн-подбором комплектующих и сопровождением пользовательских сборок. Серверная часть должна обеспечить единый каталог компонентов, хранение их характеристик, проверку совместимости, работу шаблонов, публикаций, комментариев, регистрацию покупателей и инструменты администрирования. При этом архитектура проектируется таким образом, чтобы в дальнейшем можно было

расширить её за пределы конфигуратора.

1.3 Организационная структура

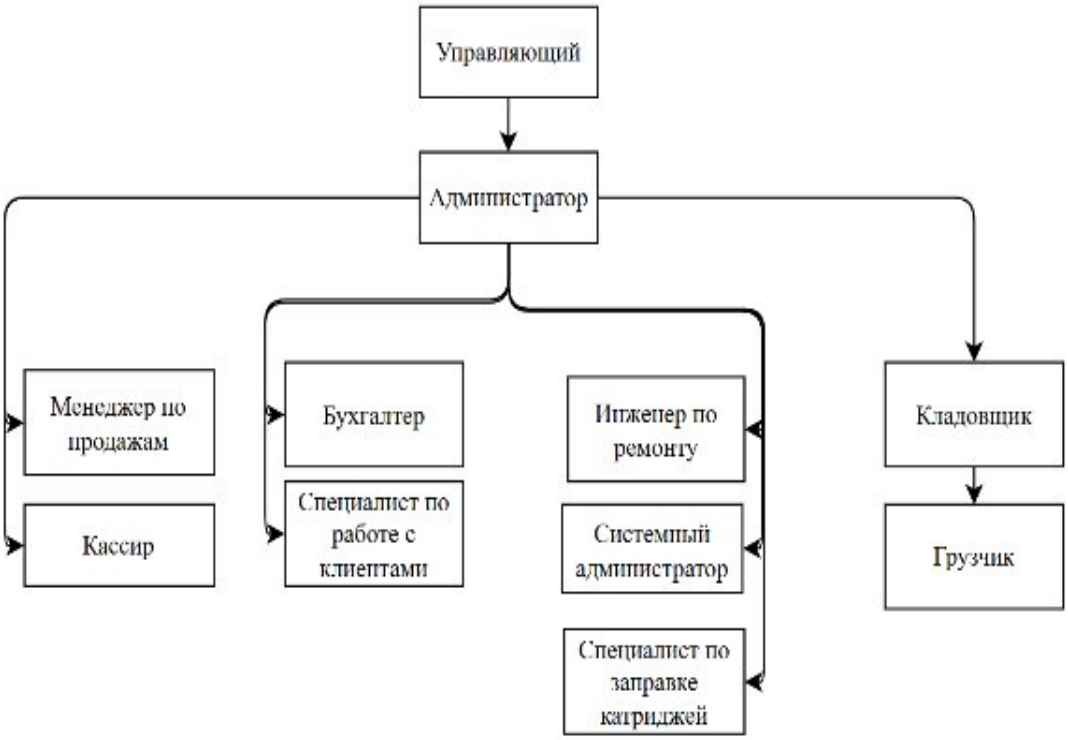

Рисунок 1 – Структура организации

Управляющий(рисунок 1) — отвечает за общую координацию работы, закупки, ценообразование и договоры с поставщиками.

Администратор — координирует работу филиалов, расписание смен, решает оперативные вопросы и контролирует сервисное обслуживание.

Менеджеры по продажам — консультирует покупателей, оформляет заказы, ведёт подбор комплектующих и выполняет план продаж.

Кассиры — выполняют операции расчёта: приём оплаты, оформление чеков, возвратов и отчётность по кассовой дисциплине.

Специалист по заправке картриджей — обслуживает и заправляет картриджи.

Инженер по ремонту — проводит диагностику и ремонт компьютеров и

оргтехники, оформляет сервисные задания.

Системный администратор — обеспечивает работу IT-инфаструктуры, локальной сети и сайта.

Кладовщик — отвечает за приёмку, хранение и выдачу товара со склада.

Грузчик — выполняет погрузочно-разгрузочные работы и перемещение товаров внутри склада.

Бухгалтер — ведёт учёт в 1С, оформляет расчёты с поставщиками и налоговую отчётность.

Специалист по работе с клиентами — обрабатывает обращения клиентов, оформляет гарантии, обеспечивает клиентскую поддержку.

1.4 Анализ реализуемых на предприятии бизнес-процессов и их связь с разрабатываемой серверной частью интернет-магазина

Закупка товаров

Цель: обеспечить оптимальный ассортимент и постоянное наличие нужных товаров для удовлетворения спроса покупателей.

Этапы и участники:

сбор отчётов по продажам в «1С:Управление торговлей» и анализ остатков. (Бухгалтер, менеджер по продажам);

формирование потребности и списка заказа. (Администратор);

утверждение заказа управляющим. (Управляющий);

согласование условий с поставщиком (цена, сроки, доставка); (Управляющий, администратор, бухгалтер);

оплата по выставленному счёту. (Бухгалтер);

организация доставки и разгрузки. (Администратор, кладовщик, грузчик);

приёмка товара на складе и внесение в учёт «1С». (Системный администратор, продавец-консультант, кладовщик).

Автоматизация:

все шаги, кроме переговоров и согласования, выполняются в «1С:Управление торговлей»;

уровень автоматизации – средний.

Значение для серверной части конфигуратора: после приёмки серверная часть должна быстро отразить изменения в доступности товаров. Если этого не сделать, пользователь будет видеть устаревшую информацию о наличии комплектующих. Поэтому серверное приложение должно поддерживать обновление остатков, изменение карточек товаров и загрузку сопутствующих изображений и характеристик.

Приёмка товаров

Цель: проверить доставленный товар по условиям заказа и зарегистрировать его на складе.

Этапы и участники:

сверка полученного груза с доставочными документами. (Администратор, кладовщик, грузчик);

разгрузка и первичная проверка целостности упаковки. (Кладовщик, грузчик);

функциональное тестирование техники. (Инженер по ремонту, продавец-консультант);

оформление акта приёмки в «1С». (Бухгалтер, кладовщик);

маркировка товаров ценниками. (Продавец-консультант, кладовщик);

размещение на складе или в торговом зале. (Продавец-консультант, кладовщик, грузчик);

Автоматизация:

сканеры штрихкодов и «1С» ускоряют ввод данных.

уровень автоматизации – средний.

Значение для серверной части конфигуратора: для корректной работы

сайта важно, чтобы пользователь видел реальные остатки. В Laravel-версии эта информация хранится в базе MySQL и используется при выборке каталога, фильтрации и отображении карточек. Следовательно, серверная часть должна обеспечивать возможность актуализации поля количества товара и исключать показ недоступных позиций как доступных к выбору.

Складской учёт и инвентаризация

Цель: вести точный учёт товарных остатков на складе и в торговом зале.

Этапы и участники:

постоянное отражение прихода и расхода товаров в «1С». (Системный администратор, продавец-консультант, кладовщик, кассир);

планирование инвентаризации (Управляющий, администратор);

физический пересчёт позиций (Продавец-консультант, кладовщик, грузчик);

сверка фактических остатков с данными учёта и выявление расхождений (Бухгалтер, кладовщик);

оформление корректировочных документов и разбор причин расхождений. (Управляющий, администратор, бухгалтер).

Автоматизация:

в магазине используются терминалы сбора данных (ТСД), поэтому пересчёт и обновление остатков проходят в «1С» сразу при сканировании;

уровень автоматизации – средний.

Значение для серверной части конфигуратора: для корректной работы сайта важно, чтобы пользователь видел реальные остатки. В Laravel-версии эта информация хранится в базе MySQL и используется при выборке каталога, фильтрации и отображении карточек. Следовательно, серверная часть должна обеспечивать возможность актуализации поля количества товара и исключать показ недоступных позиций как доступных к выбору.

Продажа товаров и обслуживание клиентов

Цель: осуществлять продажу техники и обеспечить удобное обслуживание

покупателей;

Этапы и участники:

выбор товара и консультанция (Продавец-консультант);

оформление заказа и оплата (Кассир, продавец-консультант);

подготовка товара к выдаче (Продавец-консультант, кладовщик, грузчик);

предложение дополнительных услуг (Продавец-консультант, инженер по ремонту);

фиксация продажи и передача данных в бухгалтерию (Кассир, бухгалтер).

Автоматизация:

POS-терминал и «1С:Розница» автоматизируют процесс.

Уровень автоматизации – высокий.

Консультации и сопровождение клиентов

Цель: повысить лояльность и повторные продажи.

Этапы и участники::

приём запроса от клиента лично или по телефону. (Специалист по работе с клиентами, продавец-консультант);

поиск решения и консультация. (Специалист по работе с клиентами, инженер по ремонту, продавец-консультант);

сообщение решения клиенту (Специалист по работе с клиентами).

Автоматизация:

практически весь процесс носит коммуникативный характер и мало автоматизирован. Используются лишь справочные материалы (каталоги, сайт). Специальных IT-средств нет

уровень автоматизации – очень низкий.

Значение для серверной части конфигуратора: именно низкий уровень автоматизации этого процесса делает разработку серверной части особенно

актуальной. После внедрения системы пользователь получает доступ к структурированным характеристикам, правилам совместимости, шаблонам и опубликованным сборкам, а часть консультативной нагрузки переходит от сотрудников к веб-приложению.

Подбор комплектующих и консультации клиентов

Цель: сократить нагрузку на сотрудников и дать клиенту возможность самостоятельно подобрать совместимые комплектующие.

Этапы и участники (после внедрения серверной части):

пользователь открывает конфигуратор и выбирает нужный слот.

система показывает каталог компонентов с фильтрами и поиском.

пользователь подбирает комплектующие и получает результаты проверки совместимости.

при необходимости применяется готовый шаблон.

сформированная сборка экспортируется, сохраняется или публикуется.

Автоматизация (после внедрения серверной части):

именно этот процесс в исходном состоянии был автоматизирован слабее остальных.

в рамках разрабатываемой серверной части уровень его автоматизации становится высоким, поскольку логика работы конфигуратора переносится на Laravel и MySQL.

Значение для серверной части конфигуратора: данный процесс является центральным для всего проекта. Сервер обязан поддерживать каталог компонентов, выдачу характеристик, работу шаблонов, хранение сборок, публикацию пользовательских конфигураций, голосование, комментарии и передачу собранной конфигурации в корзину.

Сервисное обслуживание и ремонт техники

Цель: осуществлять гарантийный и платный ремонт в срок.

Этапы и участники:

оформление заказа (Специалист по работе с клиентами, продавец-консультант)

диагностика неисправности (Инженер по ремонту)

согласование стоимости и сроков с клиентом (Администратор, специалист по работе с клиентами)

заказ запчастей и материалов (Бухгалтер, кладовщик)

выполнение ремонта и тестирование (Инженер по ремонту)

оформление акта выполненных работ и возврат техники клиенту (Специалист по работе с клиентами, кассир)

Автоматизация:

Заказы и статус ремонтов в «1С:Сервисный центр»;

Уровень автоматизации – средний.

1.5 Перспективы развития серверной части

Анализ бизнес-процессов показывает, что наиболее слабым с точки зрения автоматизации остаётся блок консультаций и сопровождения клиентов. Именно поэтому развитие сайта магазина должно быть направлено не только на визуальную часть конфигуратора, но и на формирование полноценной серверной платформы, которая будет поддерживать работу каталога, шаблонов, пользовательских сборок и административных функций.

Серверная часть, разработанная в рамках проекта, может стать базой для дальнейшей модернизации сайта по следующим направлениям:

интеграция с платёжными системами – подключение онлайн-оплаты (банковские карты, электронные кошельки, СБП) для автоматизации приёма платежей за сформированные сборки;

разработка публичного API – создание открытого программного интерфейса для интеграции с мобильными приложениями и сторонними сервисами;

интеграция с учётной системой «1С» – настройка двустороннего обмена данными для автоматической синхронизации товарных остатков, цен и ассортимента, что исключит необходимость ручного дублирования информации;

расширение системы совместимости – добавление на серверной стороне новых правил проверки, поддержки периферийных устройств и мониторов, а также интеллектуальных алгоритмов подбора оптимальных конфигураций;

персонализация и рекомендации – внедрение модуля рекомендаций на основе анализа истории просмотров, популярности компонентов и поведения пользователей;

визуализация сборки – реализация 3D-модели собранного системного блока с отображением расположения компонентов.

усложнение каталога сборок – добавление системы рейтингов пользователей, бейджей за опубликованные сборки, категоризации сборок по назначению (игры, работа, офис);

1.6 Анализ серверной архитектуры конкурентов

Анализ конкурентных решений проводился по следующим ресурсам:

regard.ru — магазин компьютеров и комплектующих, в конфигураторе которого можно собирать ПК, публиковать сборки и обсуждать их в комментариях. Дополнительно присутствует визуальное представление конфигурации.

citilink.ru — крупный магазин техники, где конфигуратор поддерживает создание и публикацию сборок.

dns-shop.ru — один из крупнейших ритейлеров электроники с конфигуратором, позволяющим собирать ПК, делиться результатом и обсуждать его.

torg-pc.ru — специализированный магазин с конфигуратором для подбора ПК.

pc-arena.ru — специализированный магазин компьютерных комплектующих с конструктором сборок.

gigaspot.ru — интернет-магазин, предоставляющий пользователю конфигуратор системных блоков.

hyperpc.ru — магазин премиальных компьютеров с более узким, но тщательно подобранным каталогом комплектующих.

man-made.ru — магазин готовых ПК, где представлены классический и 3D-варианты конфигуратора.

Из анализа этих решений можно сделать несколько выводов, важных именно для серверной части разрабатываемой системы:

крупные магазины строят конфигураторы на основе структурированных каталогов, где характеристики компонентов хранятся централизованно и позволяют быстро выдавать результаты поиска, фильтрации и проверки совместимости.

современные сервисы не ограничиваются только выбором комплектующих. Они поддерживают работу с пользовательскими сборками, публикацию конфигураций, комментарии, оценки, а иногда и расширенные сценарии визуализации. Следовательно, серверная часть должна обслуживать не только каталог товаров, но и пользовательский контент.

важным элементом является связка конфигуратора с актуальными остатками и ценами. Без серверного хранения и обновления таких данных конфигуратор быстро теряет практическую ценность.

конкурентные решения показывают, что серверная архитектура должна быть масштабируемой: с возможностью добавлять новые категории компонентов, новые правила совместимости и дополнительные модули поверх уже существующей базы.

Таким образом, опыт конкурентов подтверждает необходимость разработки серверной части на основе Laravel и MySQL с типизированным хранением характеристик, REST API, механизмами авторизации, поддержкой шаблонов и каталогом опубликованных сборок.

2. Технологическая часть

2.1 Постановка задачи

Наименование системы: «Серверная часть информационной системы интерактивного конфигуратора системного блока для ИП «Бит» (bit.sp)».

Назначение: создаваемая часть системы предназначена для хранения данных о комплектующих, выполнения логики конфигуратора и обслуживания запросов от браузерного интерфейса. По сути, она связывает пользовательскую часть сайта с базой данных и обеспечивает единые правила обработки информации.

Цель создания системы:

обеспечить единое и безопасное хранение сведений о товарах и их характеристиках в базе MySQL;

перенести критически важную логику с клиента на сервер: авторизацию, работу с каталогом, шаблонами и публикациями сборок;

ввести ролевую модель доступа для обычных пользователей, привилегированных сотрудников и администратора;

предоставить административные средства для управления пользователями и содержимым каталога;

реализовать модуль публичных сборок с оценками и комментариями;

использовать механизмы Laravel для защиты данных и снижения риска типовых веб-уязвимостей;

подготовить архитектуру к дальнейшему наращиванию функциональности;

повысить устойчивость и удобство работы конфигуратора за счёт серверной обработки запросов и централизованного хранения данных.

2.2 Требования к программе

Серверное приложение конфигуратора «Бит» должно реализовывать набор функций, достаточный для нормальной работы пользовательской и

административной частей сайта. В качестве платформы используется Laravel, для шаблонов страниц применяется Blade, а постоянное хранение информации выполняется в MySQL.

Модуль работы с каталогом компонентов

маршрут для получения списка товаров по категории (GET /api/products/{category}), обрабатываемый контроллером ProductController;

контроллер получает данные о товарах из базы данных через модель Product с учётом серверной пагинации (30 товаров на страницу);

реализация фильтрации по техническим характеристикам (специфичным для каждого типа компонента), по наличию на складе и по тексту поиска;

реализация сортировки (по цене по возрастанию/убыванию, по названию) на стороне сервера;

маршрут для получения доступных фильтров по категории (GET /api/products/{category}/filters) — возвращает уникальные значения характеристик;

маршрут для получения детальной информации о товаре (GET /api/product/{productId}) — возвращает товар с техническими характеристиками, списком коннекторов и, для корпусов, списком поддерживаемых размеров радиаторов;

Модуль шаблонов

маршрут для получения списка всех публичных шаблонов (GET /api/templates) — возвращает шаблоны с составом и информацией о товарах;

маршрут для создания шаблона (POST /api/templates) — авторизованный пользователь может сохранить текущую конфигурацию как шаблон;

маршрут для обновления шаблона (PUT /api/templates/{templateId}) — доступен привилегированным пользователям;

маршрут для удаления шаблона (DELETE /api/templates/{templateId}) — проверка прав доступа.

Модуль текущих сборок и корзины

маршрут для добавления сборки в корзину (POST /api/build/add-to-cart) — создаёт записи в таблицах builds, build_items и связывает с корзиной;

маршрут для экспорта сборки (POST /api/build/export) — возвращает данные сборки в формате JSON.

Модуль публичного каталога сборок

маршруты для отображения каталога сборок (GET /builds) и детальной страницы сборки (GET /builds/{id}) — Blade-шаблоны;

API-маршрут для получения списка сборок (GET /api/builds) — с поиском по названию, фильтрацией по компонентам (JSON_SEARCH), сортировкой (по новизне, по популярности), разделением на секции (подборки администрации, самые популярные);

маршрут для публикации сборки (POST /api/builds) — сохраняет данные конфигурации в формате JSON;

маршрут для голосования (POST /api/builds/{id}/vote) — реализует toggle-голосование (+1/−1) с уникальностью голоса на пару пользователь-сборка;

маршрут для комментирования (POST /api/builds/{id}/comment) — серверная валидация текста;

маршруты для модерации: удаление сборки (DELETE /api/builds/{id}), удаление комментария (DELETE /api/builds/comments/{id}) — доступны привилегированным пользователям;

кнопка «Применить сборку в конфигураторе» на странице детальной информации о сборке — передаёт состав сборки в конфигуратор через localStorage.

Модуль пользователей

страницы регистрации и входа, реализованные через Blade-шаблоны с формами и переключением вкладок;

валидация данных на стороне сервера (правила для имени, фамилии, email, пароля: минимум 8 символов, буквы в разных регистрах,

цифры);

хранение паролей в захешированном виде (bcrypt);

управление сессиями: регенерация при входе, инвалидация при выходе, регенерация CSRF-токена;

маршрут для получения текущего пользователя (GET /api/user) — используется клиентским JavaScript.

Административная панель

отдельная группа маршрутов с префиксом /admin, защищённая AdminMiddleware для проверки роли администратора;

управление пользователями — просмотр списка всех пользователей с поиском по email, изменение роли, удаление;

Blade-шаблон административной панели с таблицей пользователей.

Подсистема ролей и разграничения доступа

реализация через поле role в таблице users с допустимыми значениями: user, content_manager, warehouse_manager, admin;

реализация двух middleware: AdminMiddleware (проверка наличия привилегированной роли для административных действий) и PrivilegedMiddleware (проверка наличия роли admin, content_manager или warehouse_manager для управления контентом);

разграничение доступа: гости могут просматривать каталог компонентов и использовать конфигуратор; авторизованные пользователи дополнительно могут создавать шаблоны, экспортировать сборки, публиковать и оценивать сборки; привилегированные пользователи управляют каталогом товаров; администраторы имеют доступ к панели управления пользователями.

Требования к хранению данных

использование MySQL в качестве СУБД;

проектирование нормализованной схемы данных с типизированными таблицами для характеристик каждого типа компонента;

создание миграций для всех таблиц;

реализация сидера для первичного наполнения данными из JSON-

файлов предыдущей версии сайта;

обеспечение целостности данных (внешние ключи, каскадное удаление).

Требования к безопасности

защита от CSRF-атак (встроенная в Laravel);

хеширование паролей (bcrypt);

валидация всех входящих данных;

использование подготовленных запросов через Eloquent ORM для предотвращения SQL-инъекций;

экранирование вывода в Blade-шаблонах (автоматическое);

ограничение доступа к административным разделам и API-эндпоинтам через middleware.

Требования к производительности

серверная пагинация для списков товаров (30 элементов на страницу);

оптимизация запросов с загрузкой связей (eager loading) для минимизации обращений к базе данных.

Требования к формированию интерфейса

использование Blade-шаблонов с наследованием (общий layout для всех страниц);

дизайн страниц наследует визуальный стиль основного сайта bit.sp;

адаптивная верстка поддерживается на уровне клиентской части (CSS), сервер отдаёт корректную HTML-структуру.

Требования к тестированию

проверка корректной работы всех маршрутов и API-эндпоинтов;

тестирование сценариев: регистрация, вход, работа с каталогом, создание и применение шаблонов, публикация и оценка сборок, управление пользователями в админке;

проверка обработки ошибок (404, 403, 500);

проверка работы ролевой модели и middleware.

2.3 Разработка диаграмм

DFD (Data Flow Diagram) – уровень 0

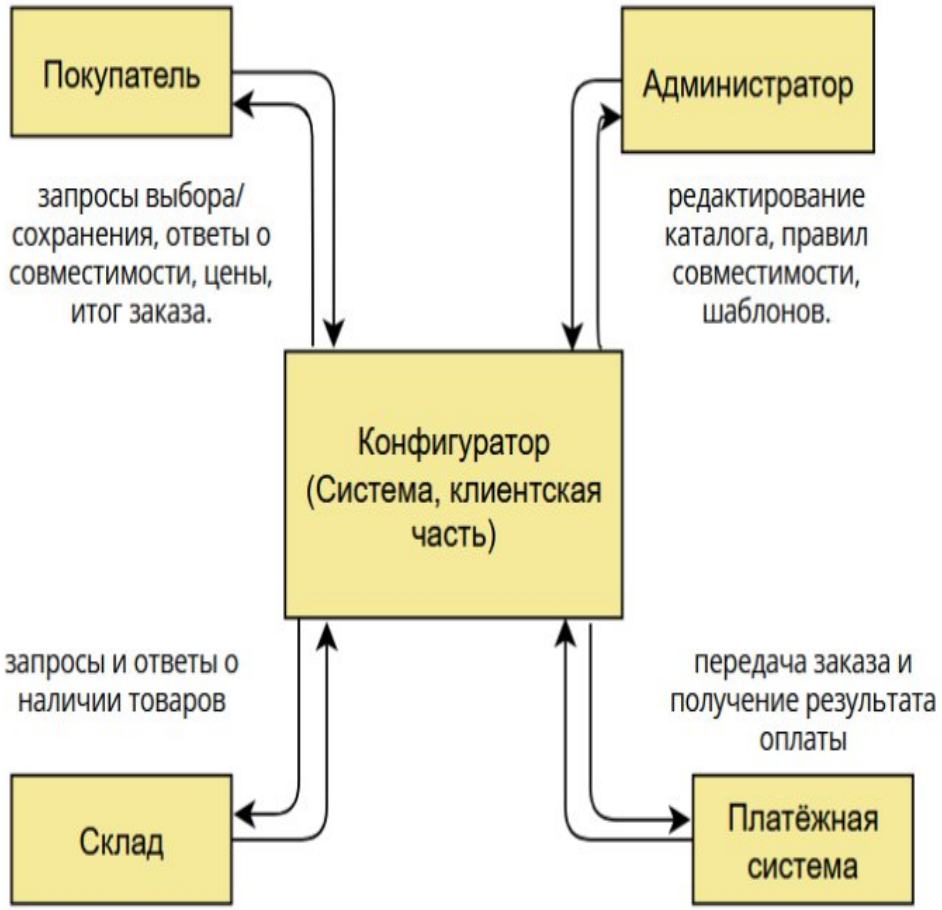

Рисунок 2 – Диаграмма DFD – уровень 0

Контекстная диаграмма потоков данных между системой и внешними сущностями(рисунок 2).

Центральный процесс: Конфигуратор - интерактивный модуль подбора сборки ПК, выполняющий выбор компонентов, проверку совместимости и формирование итоговой сборки в формате JSON для передачи в корзину/сервис оплаты.

Внешние сущности:

Покупатель (авторизованный / гость) - делает запросы выбора/сохранения, получает ответы о совместимости, цене и итого заказа; может инициировать отправку сборки в корзину;

Администратор — управляет каталогом, правилами совместимости и публичными шаблонами; может сохранять/редактировать шаблоны;

Склад — отвечает на запросы о наличии товаров (остатки), возвращает количество на складе;

Платёжная система — принимает заказ для оплаты и возвращает результат оплаты.

Ключевые потоки данных:

Покупатель -> Конфигуратор: запросы выбора компонентов, применение шаблонов, команда «Добавить в корзину»;

Конфигуратор -> Покупатель: информация о совместимости компонентов, итоговая цена, итог сборки;

Конфигуратор <-> Склад: конфигуратор запрашивает наличие компонентов, склад отвечает о их остатках;

Конфигуратор <-> Платёжная система: передача заказа для оплаты, получение результата оплаты;

Администратор -> Конфигуратор: обновления каталога/правил/шаблонов.

DFD (Data Flow Diagram) – уровень 1

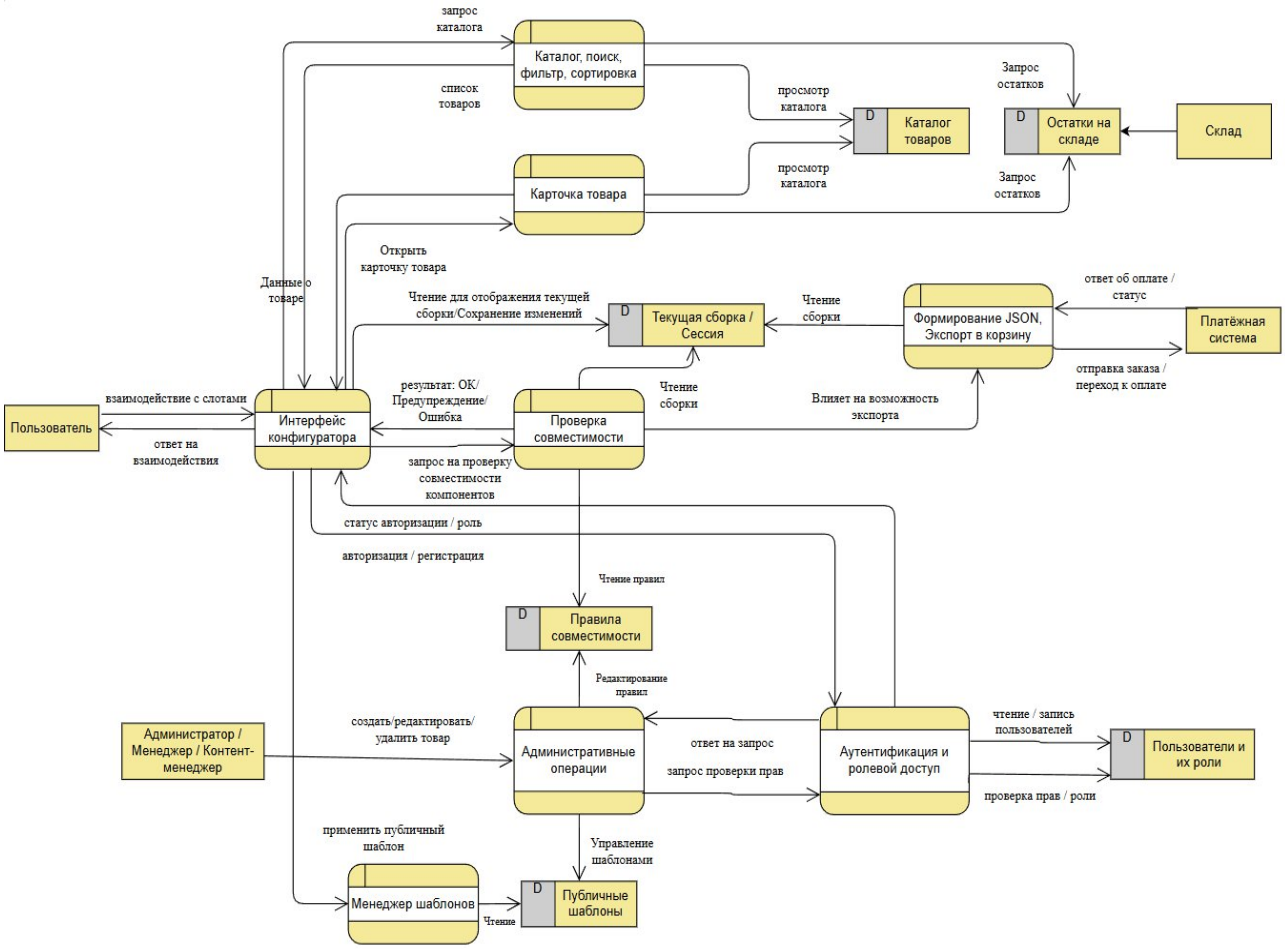

Рисунок 3 – Диаграмма DFD – уровень 1

Контекстная диаграмма потоков данных между системой и внешними сущностями(рисунок 3).

Процессы:

интерфейс конфигуратора (1.0) - то, что видит пользователь. Отсюда открываются модальные окна каталога и карточек товаров, здесь показываются слоты со смешанной информацией (выбранный компонент, краткие характеристи, цветовая индикация ошибок), блок предупреждений и итоговая панель с суммой и кнопкой «Добавить в корзину» (в зависимости от роли пользователя). Интерфейс вызывает поиск в каталоге, отображает карточку

товара, отправляет запрос на проверку совместимости и запускает экспорт в корзину. Также через интерфейс идёт регистрация/вход и применение шаблонов;

каталог / поиск / фильтры / сортировка (1.1) - механизм выдачи списка товаров: поиск по имени, фильтры (по цене, совместимости, наличию), сортировка и пагинация. Читает данные из «D1: Каталог товаров» и запрашивает остатки из D2. Возвращает в интерфейс набор карточек с минимальной информацией (id, краткое описание, цена, наличие);

карточка товара (1.2) - расширенное описание, технические характеристики, изображение, остаток и кнопка «Добавить». Получив команду «Добавить», карточка сохраняет выбор в сессию (D6). При открытии карточки состояние поиска/фильтров должно сохраниться;

движок проверки совместимости (1.3) - логика, которая принимает текущую сборку (из D6) и набор правил (из D4) и возвращает результат: OK / Предупреждение / Ошибка. Он отмечает слоты, формирует текст объяснений и влияет на возможность экспорта сборки (критические ошибки блокируют добавление в корзину/оплату);

менеджер шаблонов (1.4) - отвечает за применение существующих публичных шаблонов и за сохранение новых (если пользователь — админ). Работает с «D3: Публичные шаблоны»: читает шаблоны для пользователей и записывает/удаляет их по команде админа;

экспорт в корзину / Формирование JSON (1.5) - собирает текущую сборку из «D6: Текущая сборка / Сессия», формирует JSON с артикулом и количеством и инициирует передачу в корзину. Перед отправкой проверяет состояние совместимости у 1.3 и блокирует отправку при критических ошибках. После запроса в платёжную систему получает статус и подтверждение пользователю;

админские операции (1.6) - набор операций, доступных при наличии прав: создание и правка карточек в D1, обновление остатков в D2, управление

публичными шаблонами в D3. Перед выполнением правка проверяется через модуль аутентификации (1.7);

аутентификация и ролевой доступ (1.7) - Обрабатывает регистрацию, вход, хранение и выдачу ролей. Работает с «D5: Пользователи / Роли». Возвращает интерфейсу статус авторизации, обеспечивает видимость админских элементов и служит точкой проверки прав для 1.6.

Хранилища данных:

D1: Каталог товаров — карточки товаров: артикул, название, цена, технические характеристики, изображение;

D2: Остатки на складе — актуальные количества на складе;

D3: Публичные шаблоны — сохранённые сборки, доступные всем пользователям (имя, набор артикулов);

D4: Правила совместимости — формальные правила (сокеты, форм-фактор, TDP, требования к питанию и т. п.), которыми пользуется движок совместимости;

D5: Пользователи / Роли — аккаунты, пароли, почта, роль;

D6: Текущая сборка / Сессия — временное хранилище выбранных пользователем компонентов до экспорта; служит источником для проверки совместимости и формирования JSON.

IDEF0

Рисунок 4 – Диаграмма IDEF0

Главный процесс описывает работу клиентской части веб-конфигуратора ПК - интерактивного интерфейса для подбора комплектующих, проверки их совместимости и экспорта готовой сборки в корзину(рисунок 4)

Входы:

Каталог товаров - список продуктов с артикулами, ценами, остатками, изображениями и техническими характеристиками;

Правила совместимости - набор правил по которым проводится проверка сопоставимости компонентов;

Пользовательский ввод - действия клиента: выбор слота, фильтры, поиск, переключение страниц каталога, применение шаблонов;

Учётные данные пользователей - данные о текущем пользователе/роли (гость, авторизованный пользователь, менеджер, администратор).

Управление:

Ролевая модель - права и видимость элементов управления в зависимости от роли (гость/пользователь/менеджер склада/контент-менеджер/админ);

Правила совместимости компонентов - критерии, что считать предупреждением, а что критической ошибкой;

Требования к валидации и безопасности - правила валидации форм (регистрация, авторизация), клиентская проверка формата JSON при экспорте, требования к обработке ошибок.

Механизмы:

База данных каталога - источник данных о товарах, остатках, шаблонах;

Браузер пользователя - среда исполнения;

Администраторы и менеджеры - человеческий ресурс для наполнения каталога, управления шаблонами.

Выходы:

Интерфейс конфигуратора - визуальная страница со слотами, итоговой панелью, блоком предупреждений и модальными окнами «Каталог» / «Карточка товара»;

JSON-экспорт сборки - сериализованный пакет для передачи в корзину;

Список предупреждений и ошибок совместимости - подробно описанные сообщения с уровнями (Предупреждение / Ошибка), привязанные к слотам.

2.4 Разработка базы данных

Для работы конфигуратора требуется централизованная база данных, в

которой можно хранить как каталог комплектующих, так и пользовательские сущности, связанные со сборками. В проекте для этой цели выбрана СУБД MySQL. Схема строится с учётом нормализации, связности таблиц и возможности последующего расширения структуры без кардинальной переработки приложения.

Исходная структура данных

На начальном этапе можно представить данные в виде одной крупной таблицы, где собраны сведения о пользователях, товарах, характеристиках, шаблонах и сборках. Такой вариант удобен только как отправная точка для анализа, но не подходит для реальной эксплуатации.

Таблица 2.4.1 - Исходная таблица

Поле	Описание	Тип данных	Размер
user_id	Уникальный идентификатор пользователя	BIGINT	-
first_name	Имя пользователя	VARCHAR	64
last_name	Фамилия пользователя	VARCHAR	64
email	Электронная почта	VARCHAR	254
password	Хеш пароля	VARCHAR	255
role	Роль клиента	VARCHAR	30
product_id	Идентификатор товара	INTEGER / BIGINT	64
product_name	Наименование товара	VARCHAR	255
category_name	Название категории	VARCHAR	128
price	Цена товара	DECIMAL	10, 2
stock_quantity	Остаток на складе	INTEGER	-
main_image_path	Путь до изображения	VARCHAR	255
description	Описание товара	TEXT	-
cpu_socket	Сокет процессора	VARCHAR	32
cpu_tdp_w	TDP процессора	INTEGER	-
mb_form_factor	Форм-фактор платы	VARCHAR	32
mb_ram_slots	Число слотов RAM	INTEGER	-
ram_type	Тип памяти	VARCHAR	8
ram_size_gb	Объём планки	INTEGER	-
gpu_length_mm	Длина видеокарты	INTEGER	-
template_id	Идентификатор шаблона	VARCHAR	64
template_name	Название шаблона	VARCHAR	255
build_data	Данные сборки	JSON	-

Нормализация базы данных

Нормализация выполняется поэтапно, с переходом от исходной неструктурированной формы к схеме, соответствующей 1НФ, 2НФ и 3НФ.

Первая нормальная форма (1НФ)

Для выполнения требований первой нормальной формы каждое поле должно хранить одно неделимое значение. Поэтому повторяющиеся и множественные данные были вынесены в отдельные сущности: например, коннекторы, слоты и поддерживаемые размеры радиаторов. Характеристики разных типов компонентов также были разделены по профильным таблицам.

Вторая нормальная форма (2НФ)

На этапе приведения ко второй нормальной форме устраняются зависимости, которые не относятся ко всему ключу записи. Поскольку в исходном виде были смешаны данные о разных объектах, структура была разделена на самостоятельные, но связанные таблицы. В результате были выделены следующие сущности:

Users – информация о пользователях;

categories – категории компонентов;

products – каталог товаров;

cpus, motherboards, rams, gpus, storages, psus, coolers, cases – типизированные характеристики;

product_connectors, motherboard_slots, case_radiator_supports, product_images – дополнительные атрибуты;

templates, template_items – шаблоны сборок;

builds, build_items – рабочие сборки;

carts, cart_items – корзины;

published_builds, build_votes, build_comments – каталог опубликованных сборок.

Третья нормальная форма (3НФ)

Переход к третьей нормальной форме устраняет транзитивные зависимости. Например, сведения о категории вынесены в отдельную таблицу,

а товар хранит только ссылку на неё. Аналогичным образом остальные характеристики привязываются к идентификатору продукта, что исключает дублирование и упрощает сопровождение данных.

Итоговая структура базы данных

После завершения нормализации была получена следующая структура базы данных.

Таблица 2.4.2 - Users (пользователи)

Поле	Описание	Тип данных	Размер	Ключи
id	Уникальный идентификатор пользователя	BIGINT	-	PK
first_name	Имя	VARCHAR	64	-
last_name	Фамилия	VARCHAR	64	-
email	Электронная почта	VARCHAR	64	UNIQUE
password	Хеш пароля	VARCHAR	255	
role	Роль	VARCHAR	30	-
remember_token	Токен запоминания	VARCHAR	100	-
created_at	Дата создания	TIMESTAMP	-	-
updated_at	Дата обновления	TIMESTAMP	-	-

Таблица 2.4.3 - Categories (категории)

Поле	Описание	Тип данных	Размер	Ключи
category_id	Уникальный идентификатор	VARCHAR	32	PK
name	Название категории	VARCHAR	128	-

Таблица 2.4.4 – Products (товары)

Поле	Описание	Тип данных	Размер	Ключи
product_id	Уникальный идентификатор товара	VARCHAR	64	PK
category_id	Ссылка на категорию	VARCHAR	32	FK
sku	Артикул товара	VARCHAR	100	-
category_id	Ссылка на категорию	VARCHAR	32	-
name	Наименование	VARCHAR	255	-
price	Цена за единицу	DECIMAL	10, 2	-
stock_quantity	Количество на складе	INTEGER	-	-
description	Описание товара	TEXT	-	-
main_image_path	Путь до изображения	VARCHAR	255	-
is_active	Признак активности	BOOLEAN	-	-

Таблица 2.4.5 – Cpus (процессоры)

Поле	Описание	Тип данных	Размер	Ключи
product_id	Идентификатор товара	VARCHAR	64	PK, FK
socket	Сокет CPU	VARCHAR	32	-
tdp_w	TDP в ваттах	INTEGER	-	-
max_mem_speed	Макс. частота RAM (МГц)	INTEGER	-	-
brand	Производитель	VARCHAR	64	-
cores	Количество ядер	INTEGER	-	-
threads	Количество потоков	INTEGER	-	-
base_clock	Базовая частота (ГГц)	DECIMAL	4, 2	-
boost_clock	Турбо частота (ГГц)	DECIMAL	4, 2	-
integrated_graphics	Встроенная графика	BOOLEAN	-	-

Таблица 2.4.6 – motherboards (материнские платы)

product_id	Идентификатор товара	VARCHAR	64	PK, FK
socket	Совместимый сокет	VARCHAR	32	-
form_factor	Форм-фактор	VARCHAR	32	-
ram_slots	Число слотов RAM	INTEGER	-	-
max_ram	Макс. объём RAM (ГБ)	INTEGER	-	-
ram_speed_max	Макс. частота RAM (МГц)	INTEGER	-	-
m2_slots	Число слотов M.2	INTEGER	-	-
pcie_version	Версия PCIe	INTEGER	-	-
cpu_fan_headers	Число разъёмов CPU-FAN	INTEGER	32	-
sata_ports	Число портов SATA	INTEGER	-	-
chipset	Чипсет	VARCHAR	64	-

Таблица 2.4.7 – motherboard_slots (слоты материнской платы)

Поле	Описание	Тип данных	Размер	Ключи
id	Уникальный идентификатор	BIGINT	-	PK
product_id	Идентификатор мат. платы	INTEGER / BIGINT	64	FK
slot_type	Тип слота	VARCHAR	32	-
count	Количество	INTEGER	-	-

Таблица 2.4.8 – rams (оперативная память)

Поле	Описание	Тип данных	Размер	Ключи
product_id	Идентификатор товара	VARCHAR	64	PK, FK
ram_type	Тип	VARCHAR	8	-
size_gb	Объём планки (ГБ)	INTEGER	-	-
length_mm	Частота (МГц)	INTEGER	-	-

Таблица 2.4.9 – gpus (видеокарты)

Поле	Описание	Тип данных	Размер	Ключи
product_id	Идентификатор товара	VARCHAR	64	PK, FK
power_draw_w	Энергопотребление (Вт)	INTEGER	-	-
length_mm	Длина (мм)	INTEGER	-	-

Таблица 2.4.10 – storages (накопители)

product_id	Идентификатор товара	VARCHAR	64	PK, FK
power_draw_w	Энергопотребление (Вт)	INTEGER / BIGINT	-	FK
interface_type	Интерфейс (NVMe, SATA)	VARCHAR	16	-
pcie_version	Версия PCIe	INTEGER	-	-
capacity_gb	Объём (ГБ)	INTEGER	-	-

Таблица 2.4.11 – psus (блоки питания)

product_id	Идентификатор товара	VARCHAR	64	PK, FK
power_draw_w	Энергопотребление (Вт)	INTEGER / BIGINT	-	FK

Таблица 2.4.12 – coolers (кулеры)

product_id	Идентификатор товара	VARCHAR	64	PK, FK
cooler_height_mm	Высота (мм)	INTEGER / BIGINT	-	-
connector_pin_count	Число pin (3/4)	INTEGER	-	-
pcie_version	Версия PCIe	INTEGER	-	-
radiator_size_mm	Размер радиатора СЖО	INTEGER	-	-

Таблица 2.4.13 – cases (корпуса)

product_id	Идентификатор товара	VARCHAR	64	PK, FK
form_factor	Поддерживаемые форм-факторы	JSON	-	-
max_gpu_length_mm	Макс. длина GPU (мм)	INTEGER	-	-
max_cooler_height_mm	Макс. высота кулера (мм)	INTEGER	-	-
m2_slots	Число слотов M.2	INTEGER	-	-
drive_bays	Число отсеков для дисков	INTEGER	-	-
front_usb_c	USB-C на панели	BOOLEAN	-	-
audio_header	Аудио-разъём	BOOLEAN	-	-

Таблица 2.4.14 – product_connectors (коннекторы)

connector_id	Уникальный идентификатор	BIGINT	-	PK
product_id	Идентификатор товара	VARCHAR	64	FK
connector_type	Тип коннектора	VARCHAR	32	-

Таблица 2.4.15 – case_radiator_supports (поддержка радиаторов)

id	Идентификатор товара	VARCHAR	64	PK, FK
size_mm	Энергопотребление (Вт)	INTEGER / BIGINT	-	FK

Таблица 2.4.16 – templates (шаблоны)

template_id	Идентификатор товара	VARCHAR	64	PK, FK
name	Название шаблона	VARCHAR	255	FK
interface_type	Интерфейс (NVMe, SATA)	VARCHAR	16	-
author_user_id	Идентификатор автора	BIGINT	-	FK
is_public	Публичный / приватный	BOOLEAN	-	-

Таблица 2.4.17 – template_items (позиции шаблона)

template_item_id	Уникальный идентификатор	VARCHAR	64	PK
template_id	Идентификатор шаблона	VARCHAR	64	FK
slot_type	Тип слота	VARCHAR	32	-
product_id	Идентификатор автора	VARCHAR	64	FK

Таблица 2.4.18 – published_builds (опубликованные сборки)

id	Уникальный идентификатор	BIGINT	64	PK
user_id	Автор сборки	BIGINT	-	FK
name	Название сборки	VARCHAR	255	-
description	Описание	TEXT	-	FK
build_data	Данные сборки	JSON	-	-
total_price	Итоговая цена	DECIMAL	10, 2	-

Таблица 2.4.19 – build_votes (голоса)

id	Уникальный идентификатор	BIGINT	-	PK
published_build_id	Идентификатор сборки	BIGINT	-	FK
user_id	Идентификатор пользователя	BIGINT	-	FK
vote	Голос (+1 или −1)	TINYINT	-	-

Таблица 2.4.20 – build_comments (комментарии)

id	Уникальный идентификатор	BIGINT	-	PK
published_build_id	Идентификатор сборки	BIGINT	-	FK
user_id	Идентификатор пользователя	BIGINT	-	FK
text	Текст комментария	TEXT	-	-

ER-диаграмма

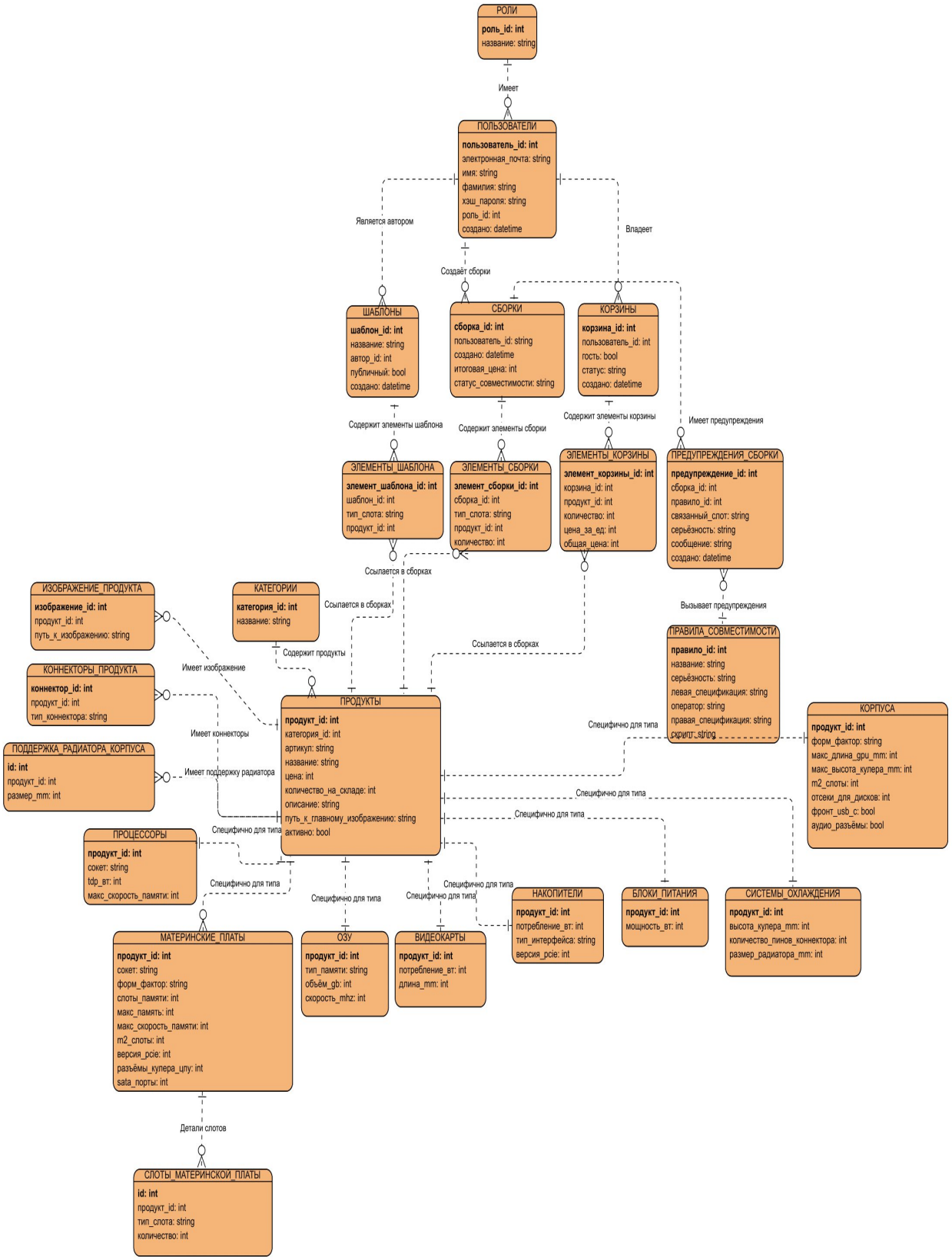

Рисунок 5 – ER-диаграмма

Обработка данных в системе происходит по следующей схеме:

пользователь заходит на сайт. Если он авторизуется, система идентифицирует его по таблице users и определяет его права по полю role;

при просмотре каталога компонентов данные загружаются из таблицы products с присоединением характеристик через типизированные таблицы (cpus, motherboards, rams и т.д.);

при работе с конфигуратором клиентская часть запрашивает данные через API, получая товары с фильтрацией и пагинацией;

при сохранении шаблона создаётся запись в таблице templates и позиции в template_items;

при экспорте сборки в корзину создаётся запись в builds и build_items, затем связь с carts через cart_items;

при публикации сборки данные конфигурации сохраняются в формате JSON в published_builds; пользователи могут оценивать (build_votes) и комментировать (build_comments);

привилегированные пользователи управляют каталогом через API: создание/редактирование/удаление записей в products и типизированных таблицах.

2.5 Реализация пользовательского интерфейса на стороне сервера

Для этого используется Blade, штатный шаблонизатор Laravel. Он позволяет формировать HTML-страницы на основе данных, которые контроллер получает из моделей и передаёт в представление.

Организация шаблонов

В проекте применена схема наследования шаблонов. Она позволяет вынести общую разметку в один базовый файл и переиспользовать её на остальных страницах. Это упрощает поддержку интерфейса и обеспечивает единый внешний вид сайта.

Базовый шаблон layouts/app.blade.php хранит повторяющуюся структуру страниц: шапку сайта, подключение стилей и скриптов, навигацию, элементы

учётной записи и нижнюю часть страницы. Через секции @yield и @stack в него подставляется содержимое конкретных экранов.

Отдельные страницы оформлены в виде дочерних шаблонов:

configurator.blade.php — основная страница конфигуратора со слотами компонентов, панелью итогов, блоком предупреждений, модальными окнами каталога, товара, редактора, шаблонов и публикации;

auth/login.blade.php — страница входа и регистрации с переключением вкладок;

admin/users.blade.php — административная панель управления пользователями;

builds/index.blade.php — каталог опубликованных сборок с поиском, фильтром по компонентам, секциями «Подборки администрации» и «Самые популярные»;

builds/show.blade.php — детальная страница сборки с компонентами, голосованием, комментариями и кнопкой «Применить в конфигураторе».

Подключение CSS и JavaScript выполняется через директиву asset(), а сами данные передаются в шаблоны методами контроллеров. Для вывода значений и условных блоков используются штатные конструкции Blade, что позволяет поддерживать шаблоны в читаемом и компактном виде.

2.6 Реализация функциональной системы

Функциональная часть системы реализована на фреймворке Laravel.

Основная логика работы авторизации и регистрации посредством контроллера AuthController:

```
public function register(Request $request)
{
    $validator = Validator::make($request->all(), [
        'first_name' => 'required|string|max:64',
        'last_name' => 'required|string|max:64',
```

```
        'email' => 'required|email|max:254|unique:users,email',
        'password' => ['required', 'string', 'min:8', 'confirmed',
            Password::min(8)->letters()->mixedCase()->numbers()],
    ]);
    $validated = $validator->validateWithBag('register');

    $user = User::create([
        'first_name' => $validated['first_name'],
        'last_name' => $validated['last_name'],
        'email' => $validated['email'],
        'password' => $validated['password'],
        'role' => 'user',
    ]);

    Auth::login($user);
    $request->session()->regenerate();
    return redirect('/');
}

public function login(Request $request)
{
    $credentials = $request->validate([
        'email' => 'required|email|max:254',
        'password' => 'required|string',
    ]);

    if (Auth::attempt($credentials, $request->boolean('remember'))) {
        $request->session()->regenerate();
        return redirect()->intended('/');
    }
```

```
    return back()->withErrors(['email' => 'Неверный email или пароль.'])
            ->onlyInput('email');
}
```

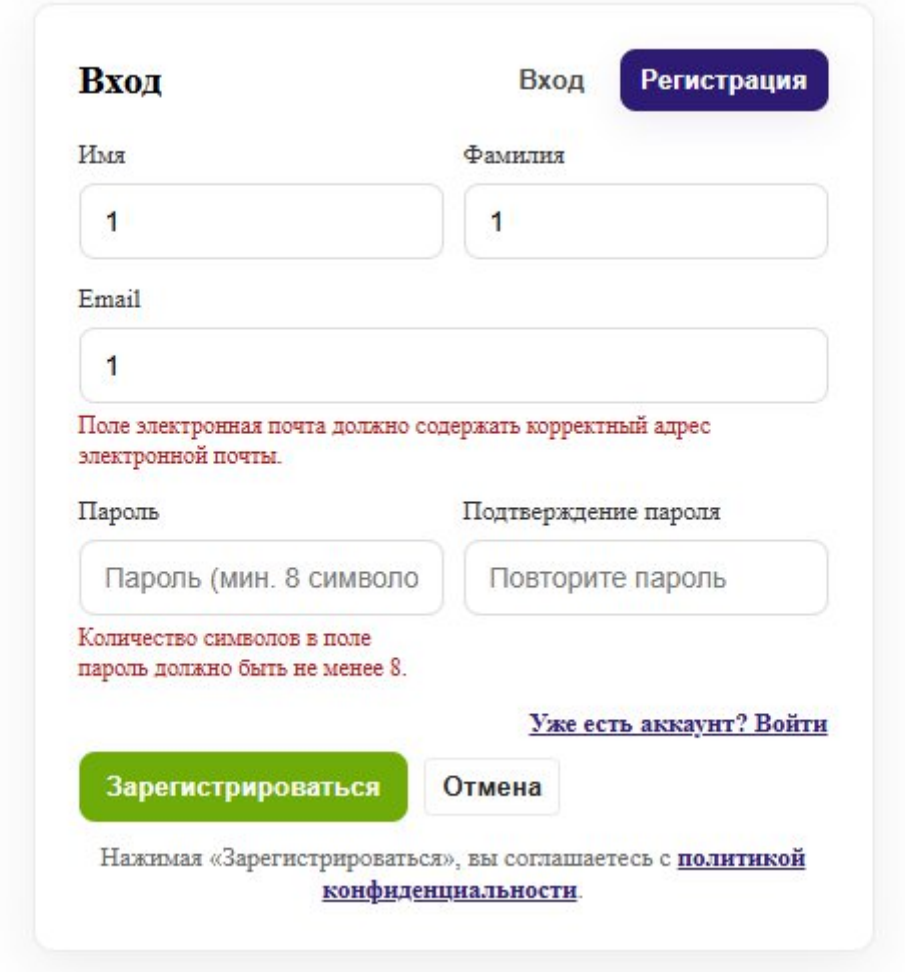

Рисунок 6 – Страница регистрации и валидация полей

Основная логика работы каталога компонентов с фильтрацией и пагинацией посредством контроллера ProductController:

```
public function byCategory(Request $request, string $category)
{
    $query = Product::where('category_id', $category)
            ->where('is_active', true);

    // Поиск по названию
    if ($search = $request->input('search')) {
```

```
            $query->where('name', 'like', '%' . $search . '%');
        }

        // Фильтр по наличию
        if ($request->boolean('in_stock')) {
            $query->where('stock_quantity', '>', 0);
        }

        // Фильтрация по характеристикам (spec filters)
        $specTable = $this->specTable($category);
        if ($specTable) {
            $query->join($specTable,                'products.product_id',                '=',
"$specTable.product_id");
            foreach ($request->except(['search','sort','limit','page','in_stock']) as $col
=> $val) {
                if ($val !== null && $val !== '') {
                    $values = array_map('trim', explode(',', $val));
                    $query->whereIn("$specTable.$col", $values);
                }
            }
        }

        // Сортировка
        $sort = $request->input('sort', 'name');
        match ($sort) {
            'price_asc'  => $query->orderBy('price', 'asc'),
            'price_desc' => $query->orderBy('price', 'desc'),
            'name'       => $query->orderBy('name', 'asc'),
            default      => $query->orderBy('name', 'asc'),
        };
```

```
        $limit = min((int)($request->input('limit', 30)), 200);
        return response()->json($query->paginate($limit));
    }
```

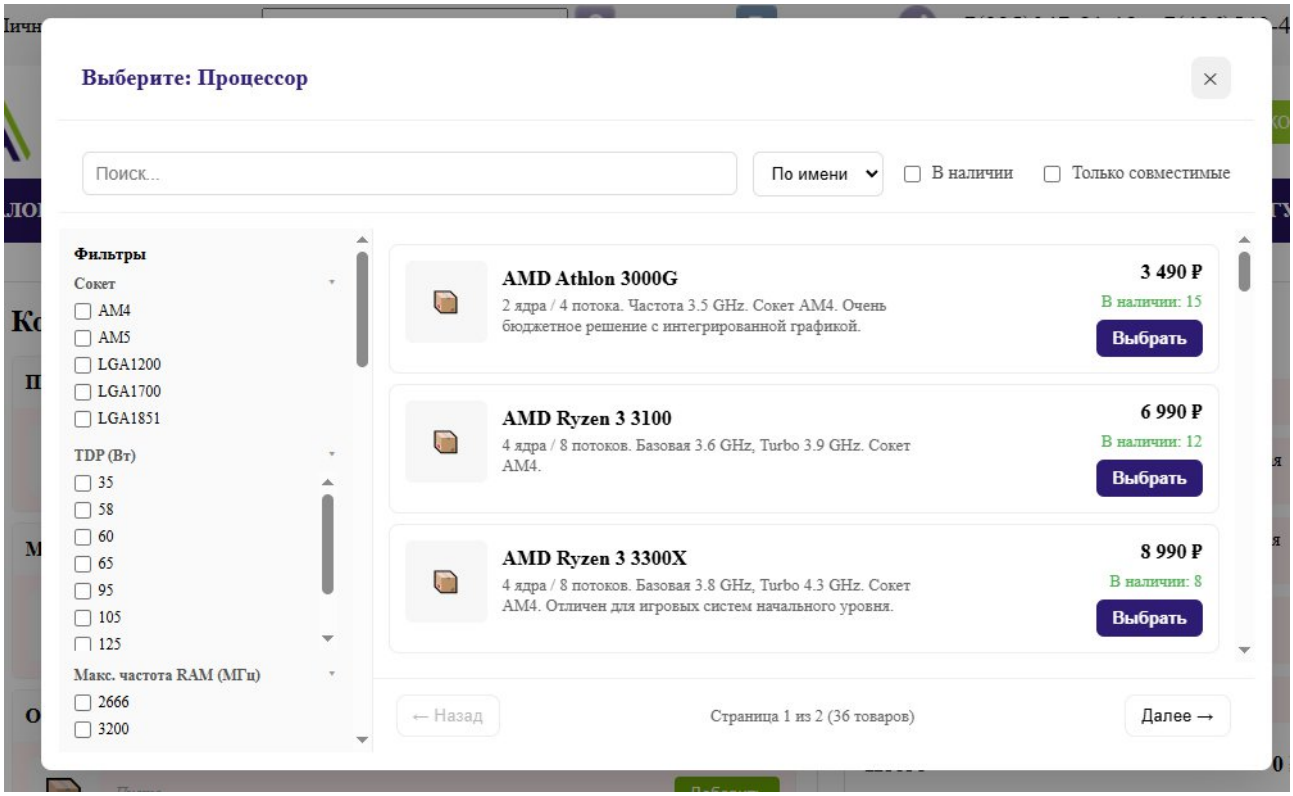

Рисунок 7 – Каталог компонентов с фильтрацией в конфигураторе

Основная логика работы каталога опубликованных сборок посредством контроллера BuildCatalogController:

```
    public function apiIndex(Request $request)
    {
        $query = PublishedBuild::with('user')
         ->selectRaw('published_builds.*, (SELECT COALESCE(SUM(bv.vote), 0)
                FROM build_votes bv
                WHERE bv.published_build_id = published_builds.id) as score');

        // Поиск по названию
        if ($search = $request->input('search')) {
            $query->where('name', 'like', '%' . $search . '%');
```

```
        }

        // Фильтр по компонентам
        if ($component = $request->input('component')) {
            foreach (explode(',', $component) as $pid) {
                $pid = trim($pid);
                if ($pid !== '') {
                    $query->whereRaw(
                        "JSON_SEARCH(published_builds.build_data,        'one',        ?,
'\$.*.product_id') IS NOT NULL",
                        [$pid]
                    );
                }
            }
        }

        // Секции и сортировка
        $section = $request->input('section', '');
        if ($section === 'admin') {
            $query->whereHas('user', fn($q) =>
            $q->whereIn('role', ['admin','content_manager','warehouse_manager']));
        }

        $sort = $request->input('sort', 'newest');
        if ($sort === 'popular') {
            $query->orderByDesc('score');
        } else {
            $query->orderByDesc('created_at');
        }

        return response()->json($query->paginate($request->input('limit', 12)));
```

```
}
```

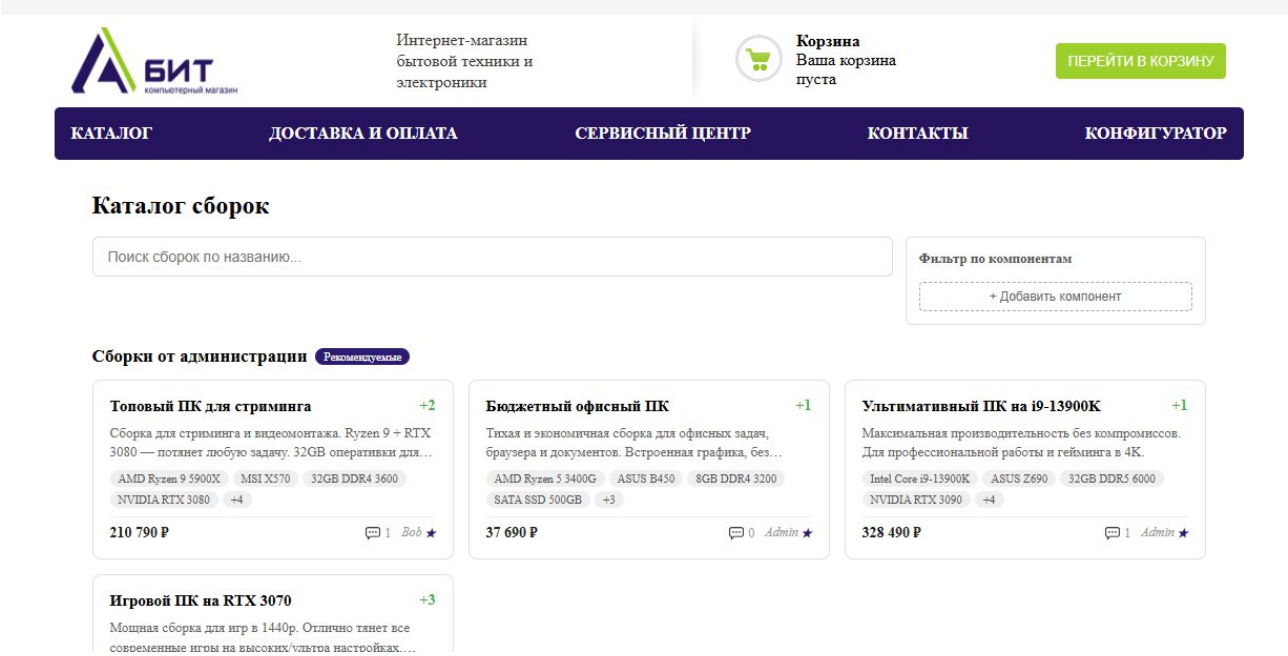

Рисунок 8 – Каталог опубликованных сборок

Логика голосования за сборки:

```
public function vote(Request $request, int $id)
{
    $request->validate(['vote' => 'required|integer|in:-1,1']);
    $existing = BuildVote::where('published_build_id', $id)
                    ->where('user_id', Auth::id())->first();

    if ($existing) {
        if ($existing->vote === (int)$request->vote) {
            $existing->delete();
        } else {
            $existing->update(['vote' => $request->vote]);
        }
    } else {
        BuildVote::create([
            'published_build_id' => $id,
            'user_id' => Auth::id(),
```

```
            'vote' => $request->vote,
        ]);
    }

    $score = BuildVote::where('published_build_id', $id)->sum('vote');
    $userVote = BuildVote::where('published_build_id', $id)
                    ->where('user_id', Auth::id())->value('vote');
    return response()->json(['score' => $score, 'user_vote' => $userVote]);
}
```

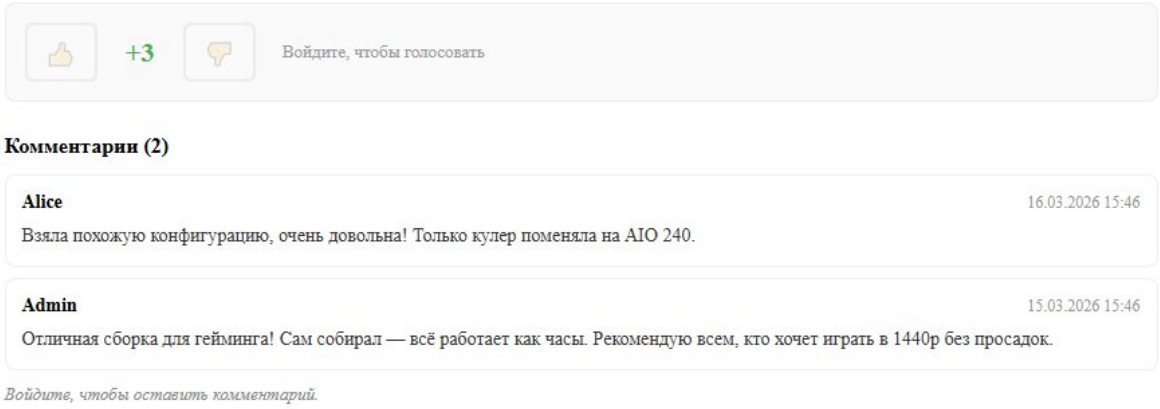

Рисунок 9 – Система голосования и комментариев

2.7 Тестирование системы

Таблица 2.7.1 – Тест кейсы

ID	Название теста	Шаги / входные данные	Ожидаемый результат	Фактический результат
TC-01	Регистрация нового пользователя (валидные данные)	Заполнить форму регистрации: имя, фамилия, email, пароль (мин. 8 символов, буквы в разных регистрах, цифры), подтверждение пароля	Успешная регистрация; пользователь автоматически авторизован и перенаправлен на главную	Совпадает с ожидаемым
TC-02	Регистрация — валидация пароля (слабый)	Ввести пароль «123456»	Показывается ошибка валидации пароля	Совпадает с ожидаемым
TC-03	Вход — успешный	Ввести существующие в системе email и пароль	Успешная авторизация, перенаправление на конфигуратор	Совпадает с ожидаемым
TC-04	Вход — неверные данные	Ввести несуществующие email/пароль	Показывается ошибка «Неверный email или пароль»	Совпадает с ожидаемым

Продолжение таблицы 2.7.1 – Тест кейсы

TC-05	Добавление компонента в слот конфигуратора	Нажать на слот, выбрать компонент в каталоге	Компонент отображается в слоте, итоговая цена обновлена	Совпадает с ожидаемым
TC-06	Проверка совместимости (несовпадение сокетов)	Выбрать CPU с socket AM4 и MB с socket LGA1700	Слоты подсвечены красным, в блоке предупреждений ошибка, экспорт заблокирована пуста".	Совпадает с ожидаемым
TC-07	Применение шаблона	Нажать «Применить» на шаблоне из списка	Все слоты заполнены компонентами шаблона, проверка совместимости запущена	Совпадает с ожидаемым
TC-08	Публикация сборкии	Собрать конфигурацию, нажать «Опубликовать», ввести название	Сборка появляется в каталоге сборок	Совпадает с ожидаемым
TC-09	Голосование за сборку	На странице сборки нажать голос за	Счётчик увеличивается на 1,	Совпадает с ожидаемым

2.8 Руководство пользователя

Ниже приведён краткий порядок работы с системой для обычного пользователя.

Откройте сайт bit.sp — откроется страница конфигуратора;

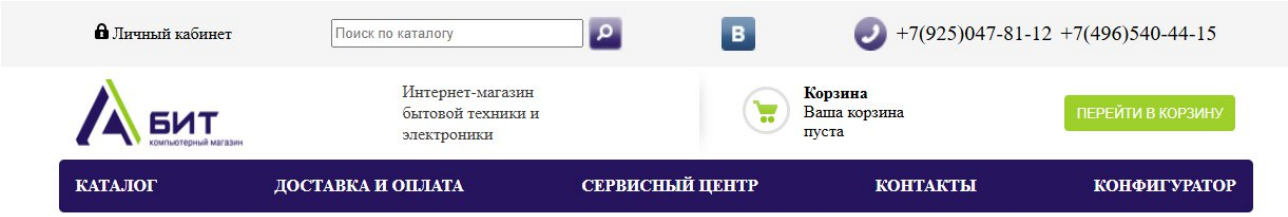

Рисунок 10 – Шапка сайта с кнопкой «Личный кабинет»

Для регистрации нажмите «Личный кабинет» в шапке, затем заполните форму: имя, фамилия, email, пароль и подтверждение пароля;

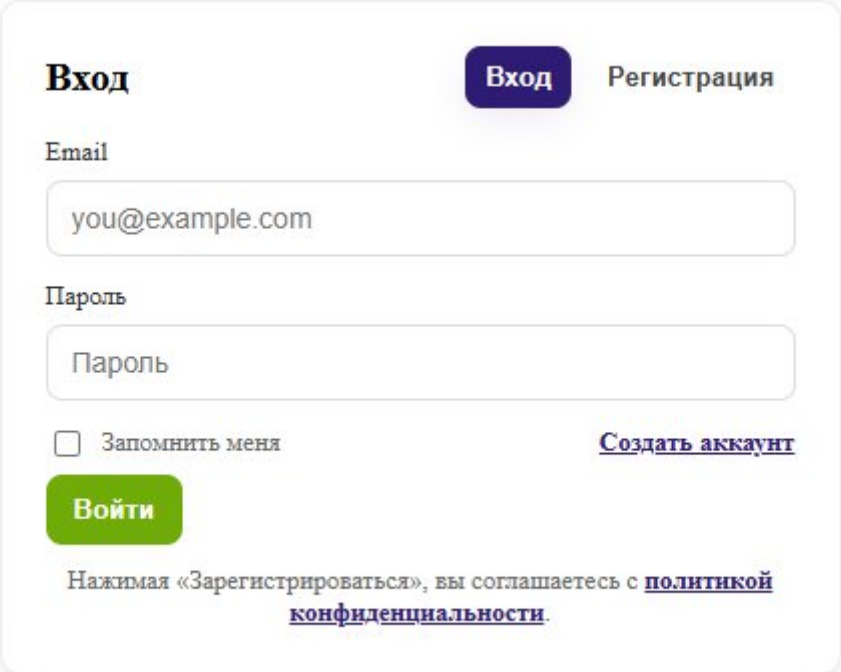

Рисунок 11 – Форма входа

Если вы уже зарегистрированы — перейдите на вкладку «Вход», введите email и пароль;

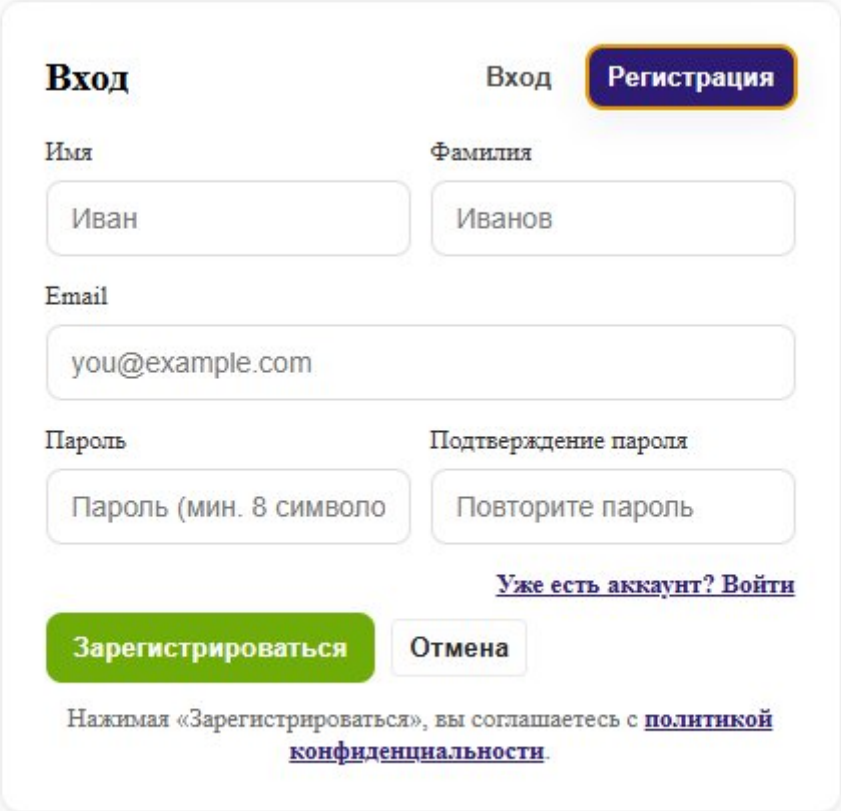

Рисунок 12 – Форма регистрации

Основная рабочая область конфигуратора содержит: список слотов для компонентов (процессор, материнская плата, оперативная память, видеокарта, накопитель, блок питания, кулер, корпус), итоговую панель с общей ценой и кнопками экспорта/публикации, блок предупреждений о совместимости;

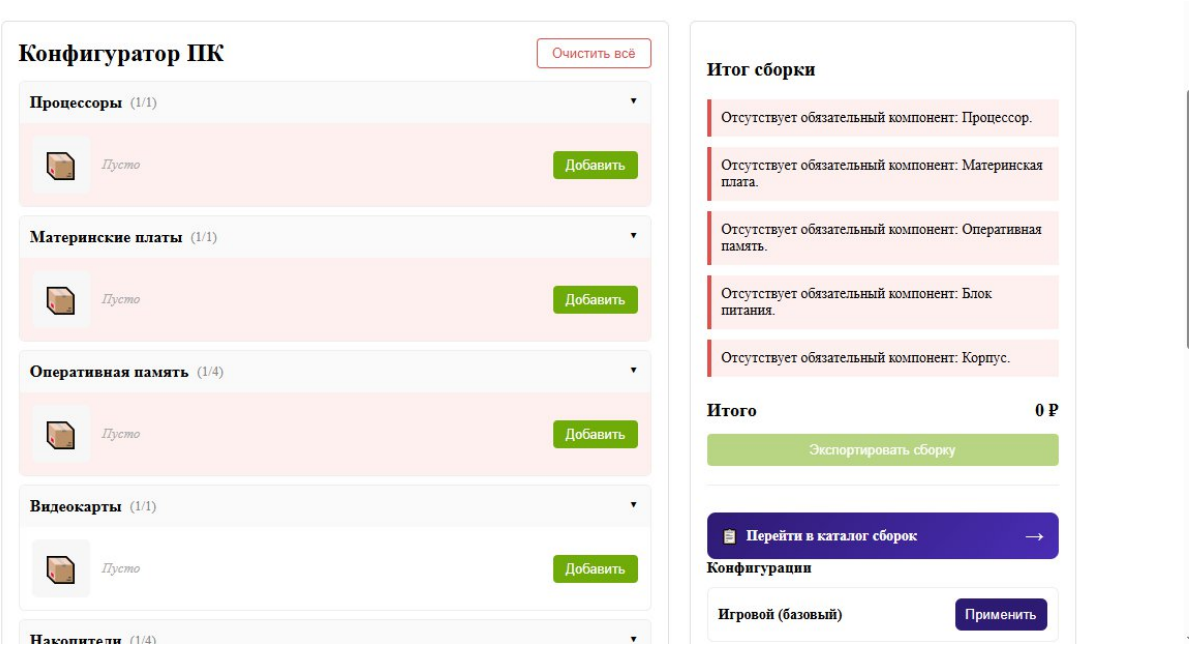

Рисунок 13 – Основная рабочая область конфигуратора

Нажмите кнопку «Выбрать» в нужном слоте — откроется модальное окно каталога компонентов с поиском, фильтрацией по характеристикам, сортировкой и пагинацией;

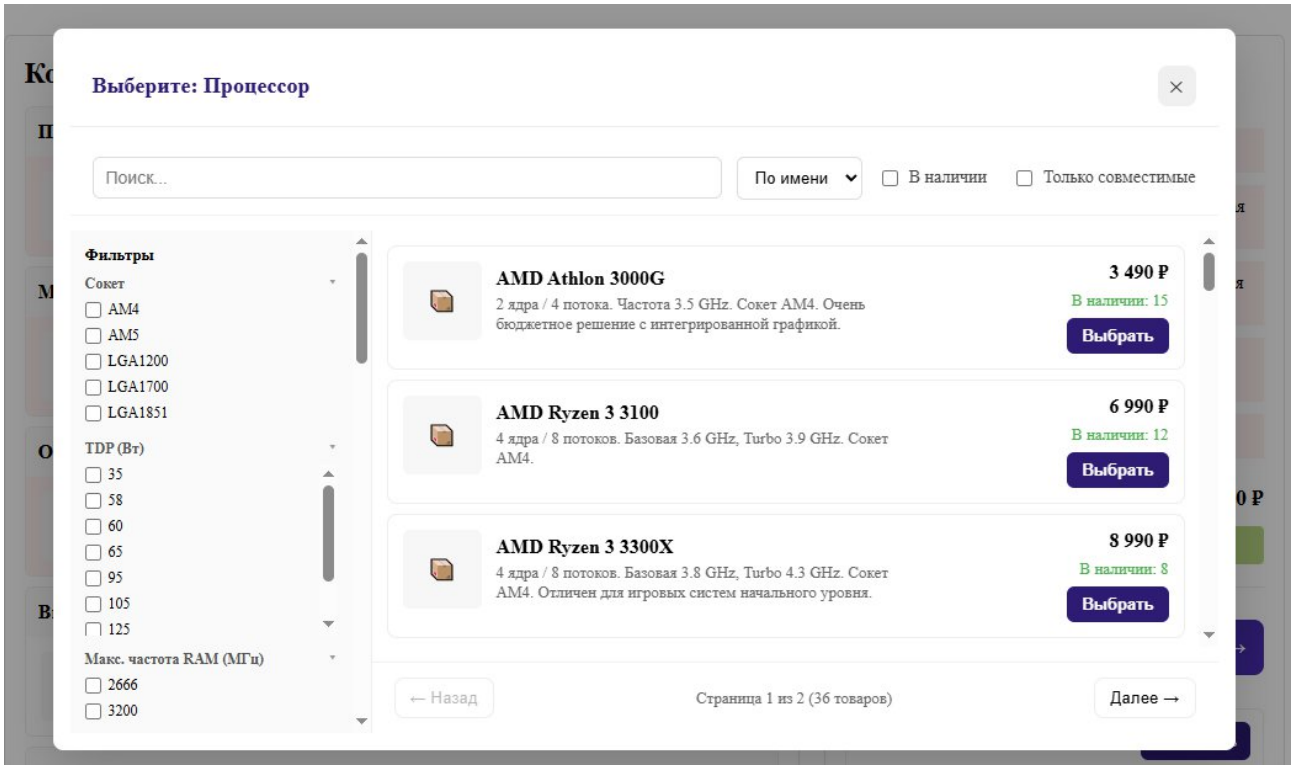

Рисунок 14 – Модальное окно каталога компонентов

Для просмотра подробностей кликните на карточку товара — откроется детальное описание с характеристиками, изображением и кнопкой «Выбрать»;

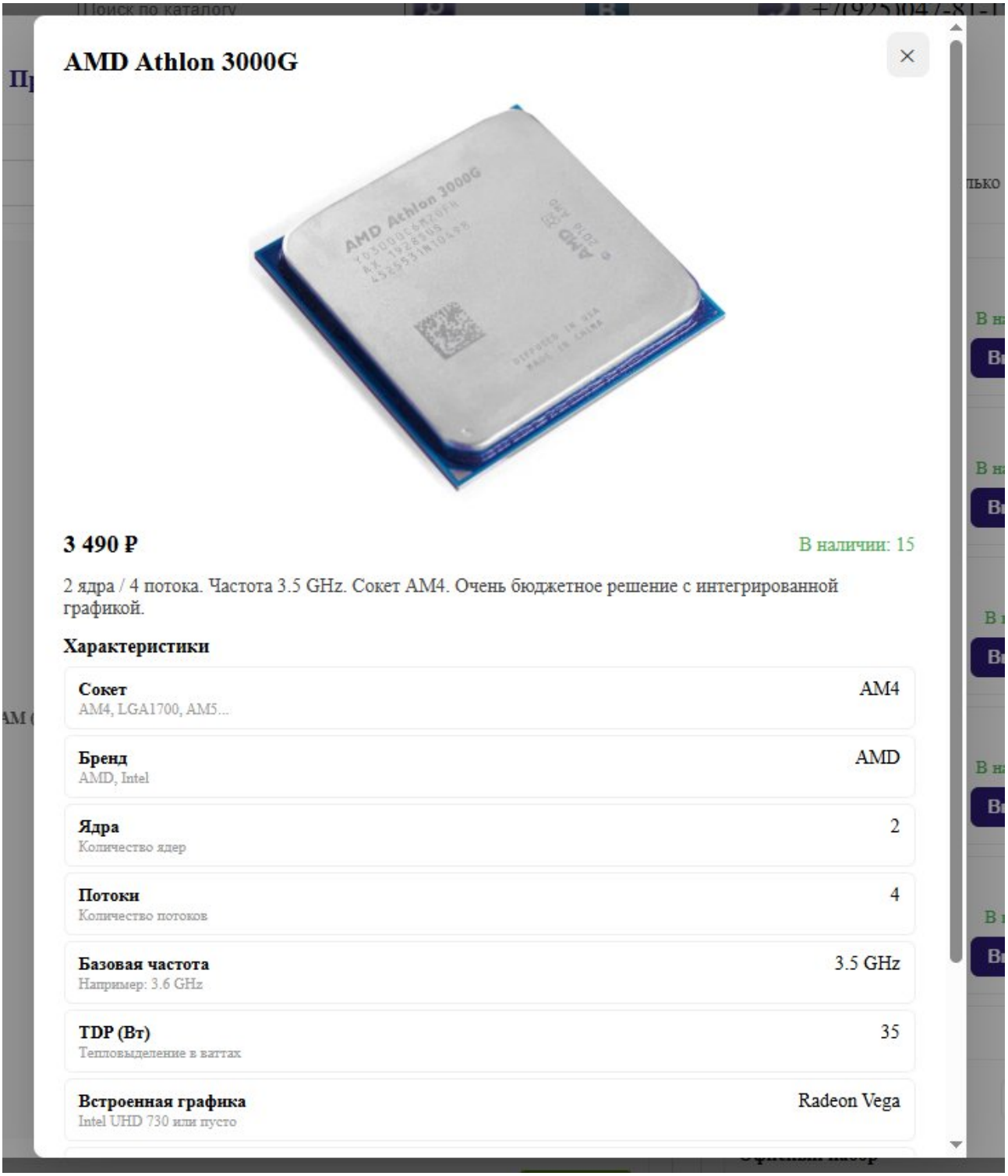

Рисунок 15 – Карточка товара

После выбора компонента система автоматически проверит совместимость; проблемные слоты подсвечиваются, сообщения отображаются в блоке предупреждений;

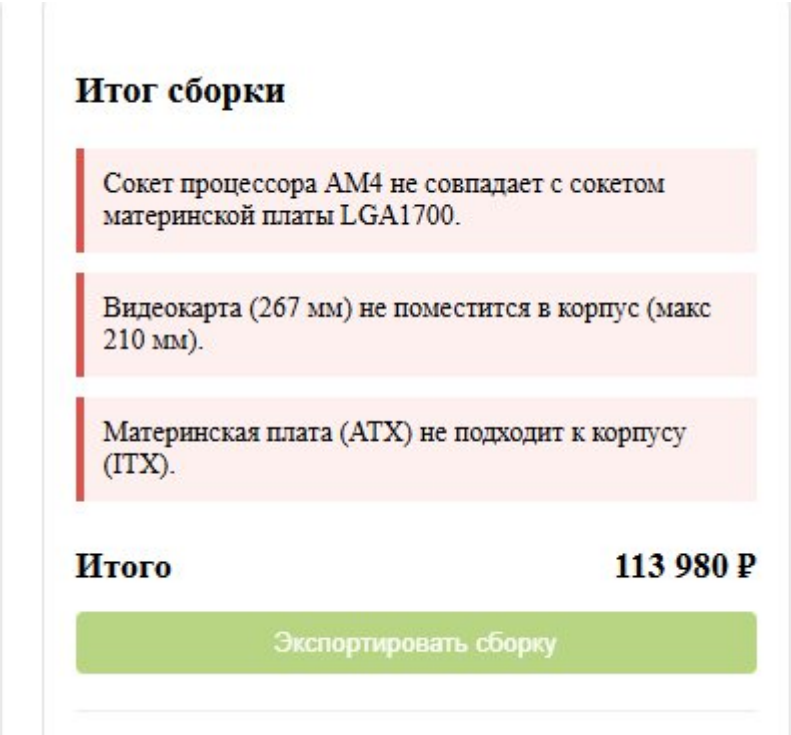

Рисунок 16 – Демонстрация проверки совместимости

В боковой панели доступны готовые шаблоны сборок — нажмите «Применить», чтобы заполнить конфигуратор предустановленным набором компонентов;

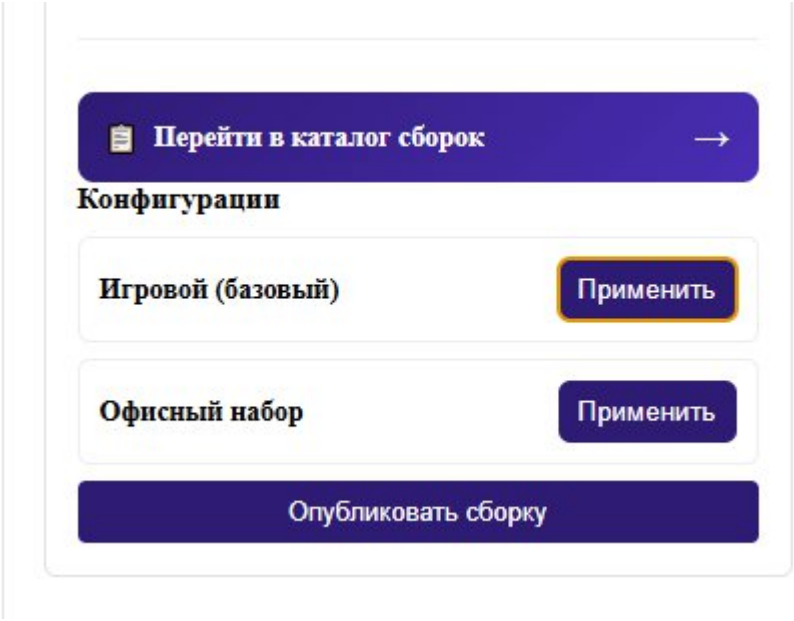

Рисунок 17 – Панель шаблонов

Для публикации сборки нажмите «Опубликовать сборку», введите название и описание — сборка появится в каталоге сборок;

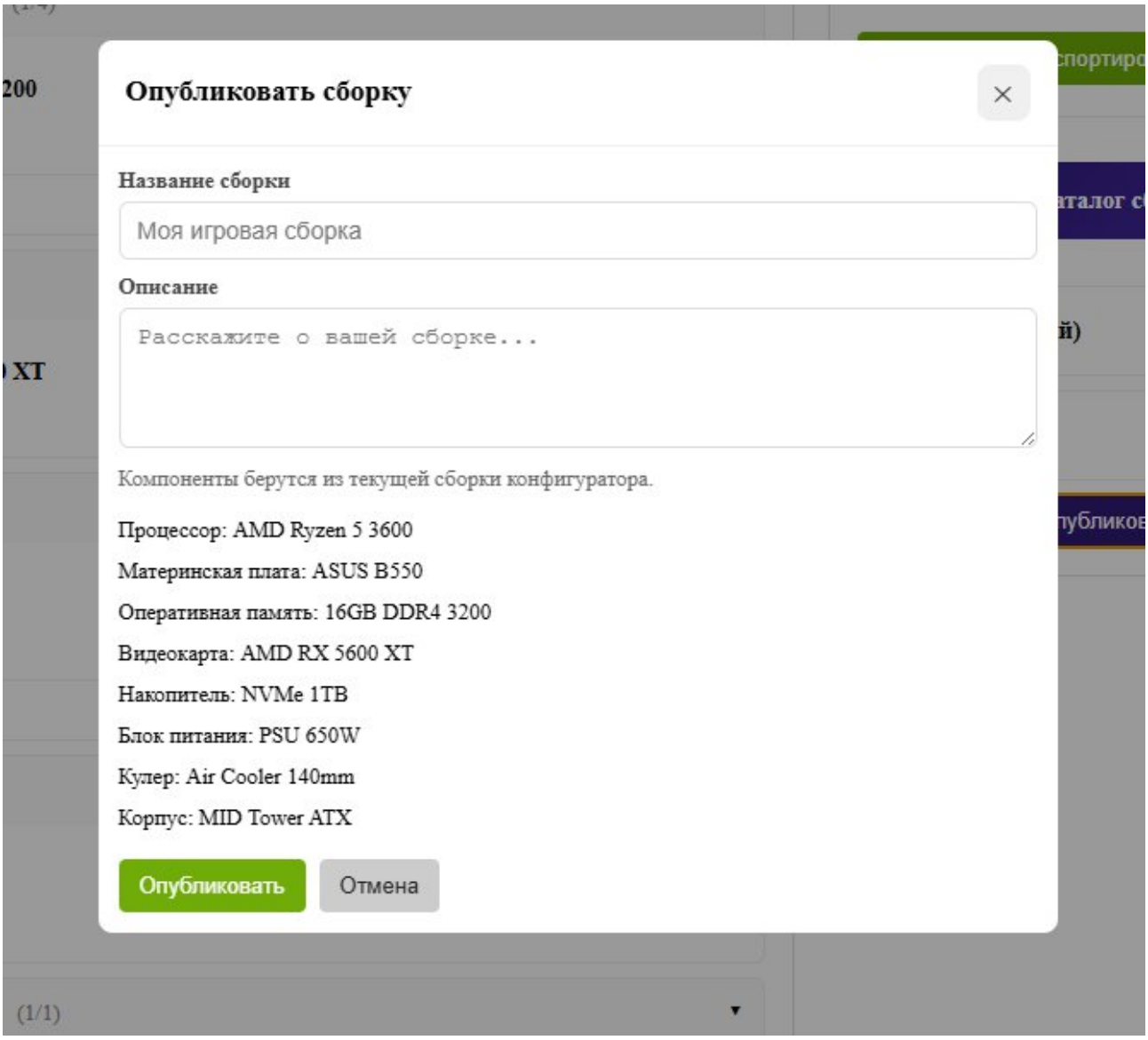

Рисунок 18 – Модальное окно публикации сборки

В каталоге сборок (ссылка в конфигураторе) можно просматривать сборки других пользователей, голосовать и оставлять комментарии;

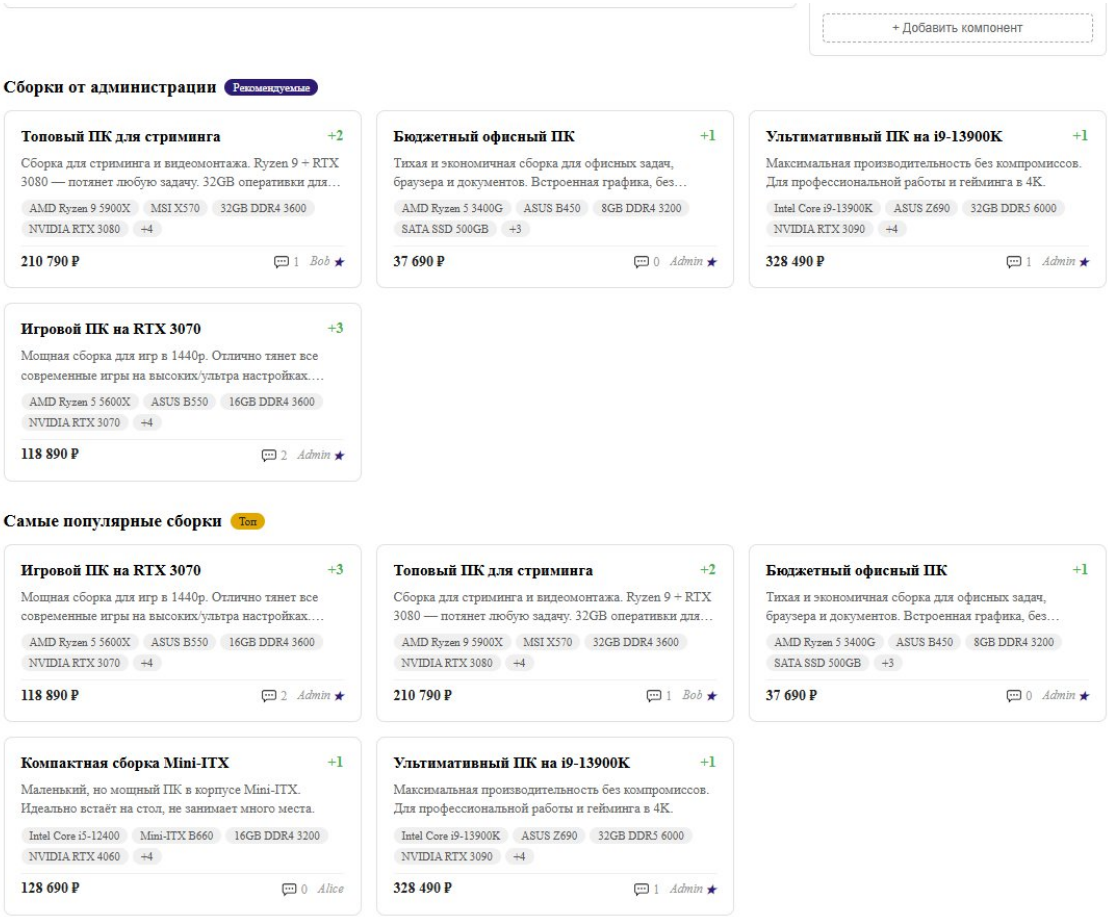

Рисунок 19 – Каталог опубликованных сборок

На странице сборки можно нажать «Применить сборку в конфигураторе», чтобы загрузить все компоненты в свой конфигуратор;

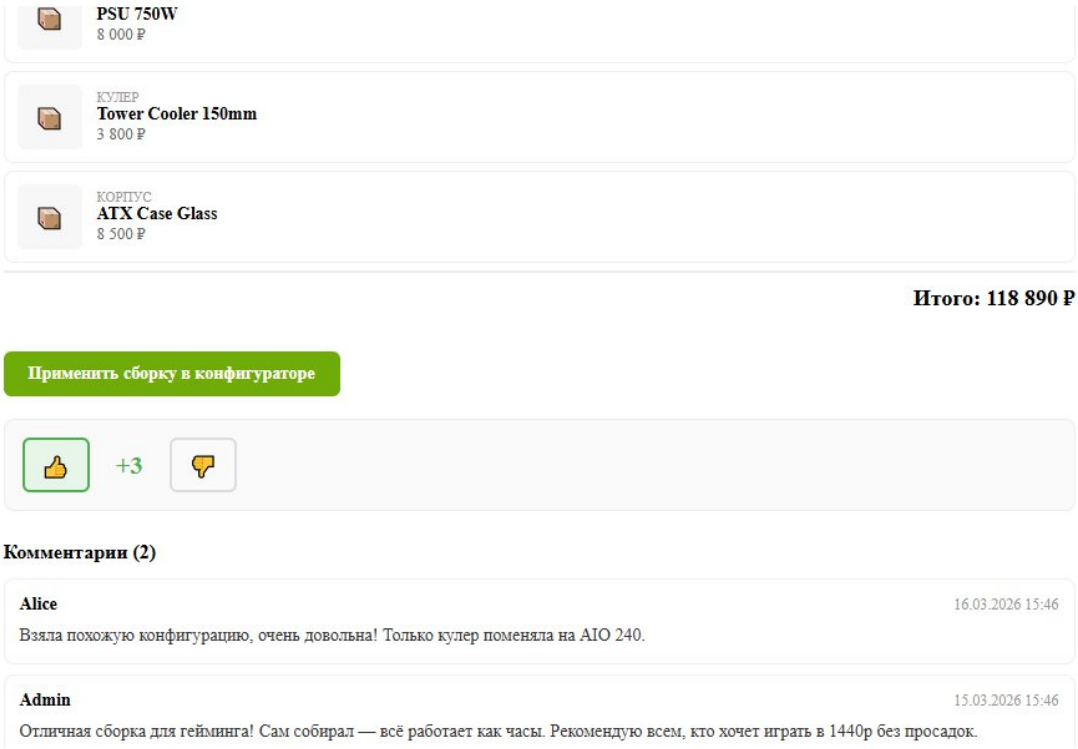

Рисунок 20 – Детальная страница сборки

Порядок работы для администратора и привилегированных сотрудников имеет следующий вид.

Войдите под учётной записью с ролью администратор, контент-менеджер или менеджер склада;

В конфигураторе появятся дополнительные элементы управления: кнопки «Добавить товар», «Редактировать» и «Удалить» в каталоге компонентов;

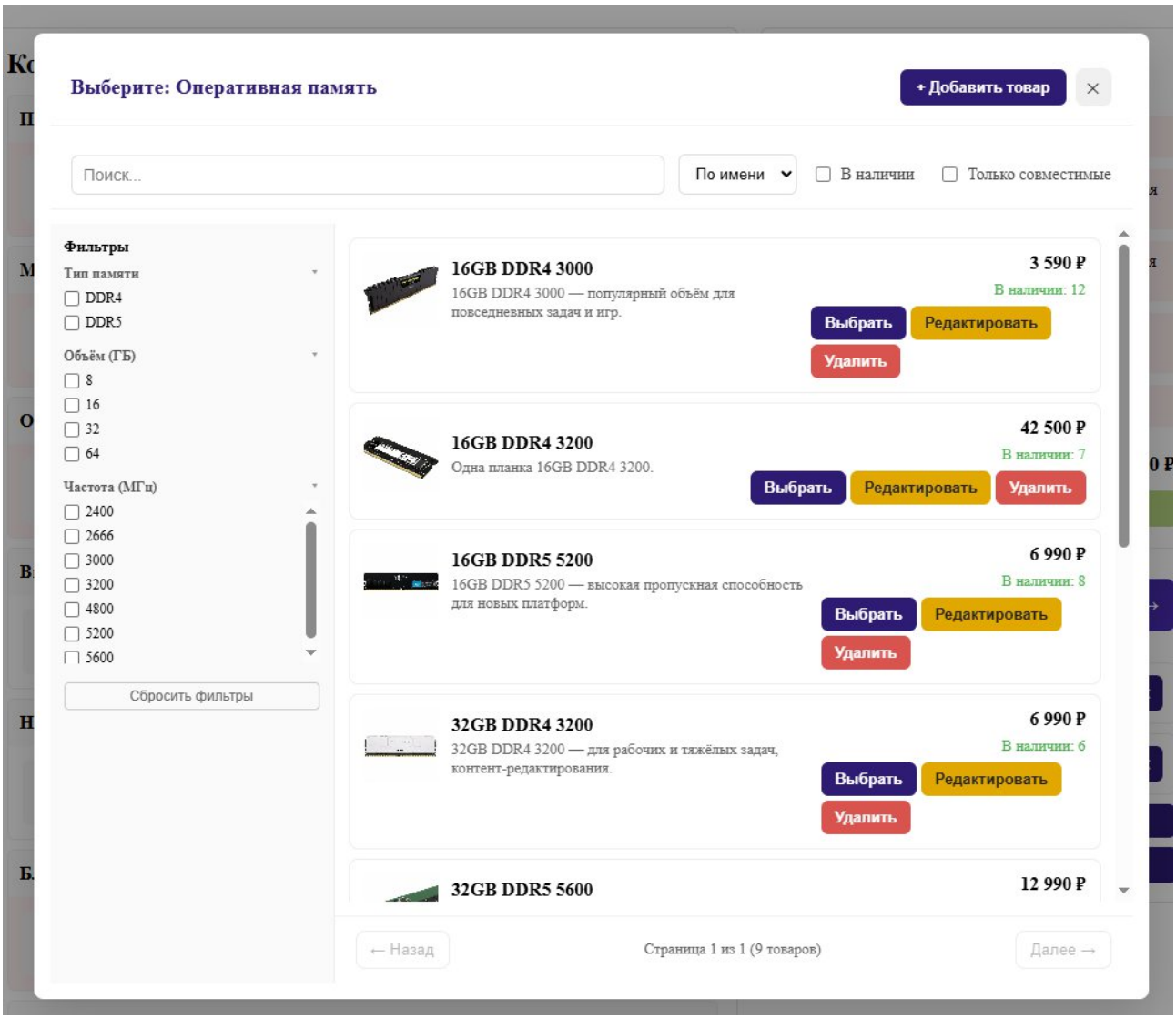

Рисунок 21 – Административные элементы в каталоге

Для добавления товара нажмите «Добавить товар» в каталоге — откроется форма с полями характеристик, специфичных для выбранной категории;

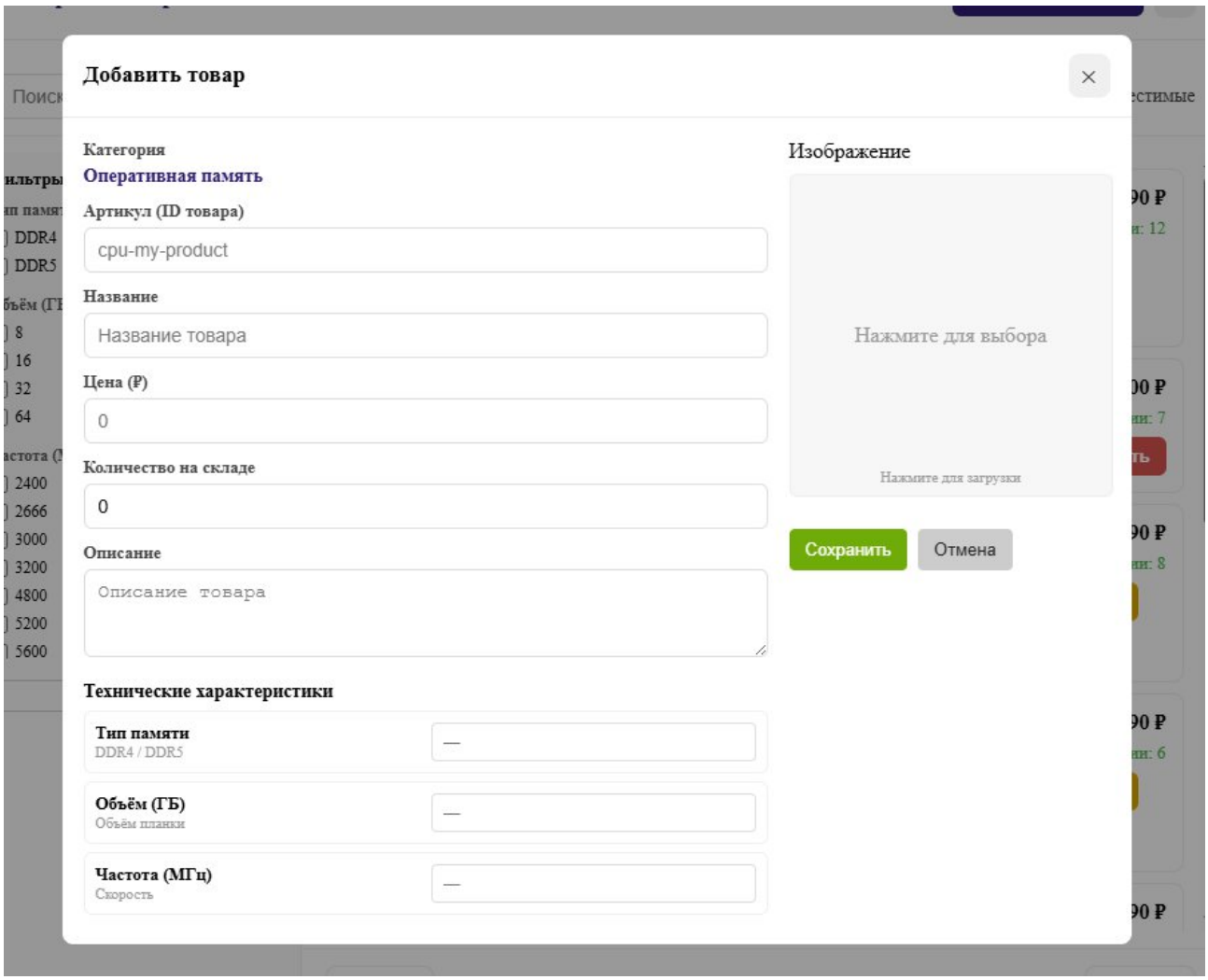

Рисунок 22 – Форма добавления товара

Для управления пользователями (только администратор) перейдите по кнопке в шапке на страницу «Управление пользователями» — здесь можно менять роли и удалять аккаунты;

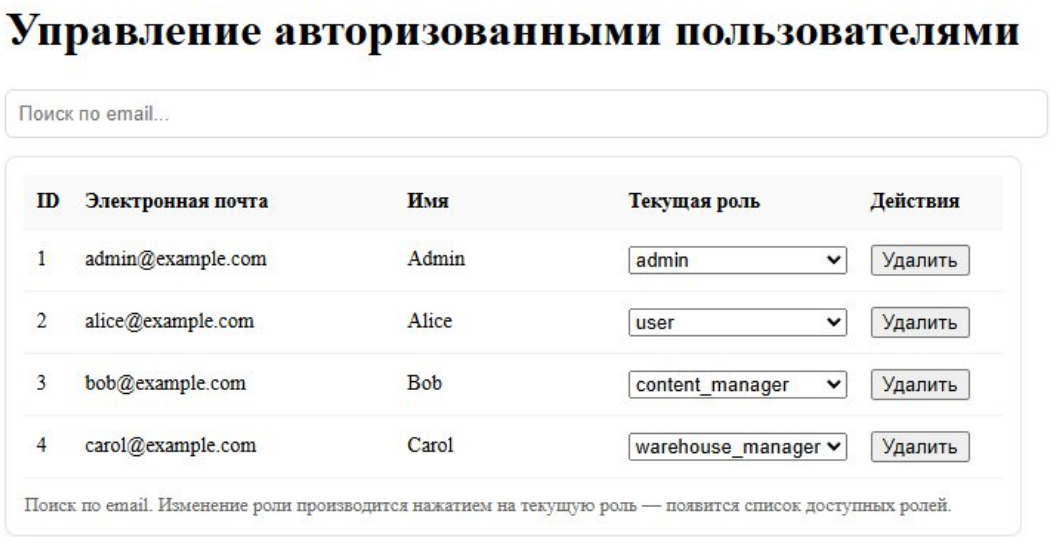

Рисунок 23 – Административная панель управления пользователями

Привилегированные пользователи могут модерировать каталог сборок: удалять сборки и комментарии;

Рисунок 24 – Модерация каталога сборок

Заключение

В ходе выполнения курсового проекта была создана серверная часть для конфигуратора системного блока магазина «Бит». Выполненная работа позволила перевести проект от статического клиентского прототипа к архитектуре, в которой основные данные и правила обработки сосредоточены на сервере.

В процессе разработки была проанализирована предметная область, определены требования к функциональности, спроектирована база данных и реализовано приложение на Laravel. Серверная часть охватывает хранение каталога комплектующих, учёт технических характеристик, регистрацию и вход пользователей, управление ролями, работу с шаблонами, экспорт и публикацию сборок, а также взаимодействие с каталогом опубликованных конфигураций.

Важным результатом стало расширение возможностей по сравнению с предыдущей версией сайта. Вместо хранения данных в JSON используется MySQL, вместо клиентской авторизации применяется серверная аутентификация, а логика системы стала поддерживать многослотовые компоненты, расширенные правила совместимости, систему комментариев и голосов, административное управление и динамическое формирование страниц.

Проведённая проверка показала, что реализованные функции работают корректно в основных сценариях использования. Разработанная архитектура остаётся пригодной для дальнейшего развития: к ней можно подключить внешние платёжные сервисы, синхронизацию с учётными системами, дополнительные типы оборудования и новые инструменты подбора конфигураций.

Библиография

1. ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.

2. ГОСТ 2.105–2019. ЕСКД. Общие требования к текстовым документам.

3. Laravel Documentation. The PHP Framework for Web Artisans. — URL: https://laravel.com/docs (дата обращения: 15.03.2026).

4. MySQL Documentation. MySQL Reference Manual. — URL: https://dev.mysql.com/doc/ (дата обращения: 15.03.2026).

5. PHP Manual. — URL: https://www.php.net/manual/ (дата обращения: 15.03.2026).

6. MDN Web Docs. HTTP. — URL: https://developer.mozilla.org/ru/docs/Web/HTTP (дата обращения: 15.03.2026).

7. OWASP Foundation. OWASP Top Ten. — URL: https://owasp.org/www-project-top-ten/ (дата обращения: 15.03.2026).

8. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2021. — 448 с.

9. Stauffer M. Laravel: Up & Running: A Framework for Building Modern PHP Apps. — 3rd ed. — O'Reilly Media, 2023. — 520 p.

10. Дакетт Дж. PHP и MySQL: разработка веб-приложений. — 5-е изд. — СПб.: Питер, 2022. — 720 с.

11. Григорьев А.В. Проектирование баз данных. Нормализация и ER-моделирование. — М.: ИНТУИТ, 2021. — 240 с.

12. Zandstra M. PHP 8 Objects, Patterns, and Practice. — 7th ed. — Apress, 2024. — 800 p.

13. Laravel News. Community tutorials and packages. — URL: https://laravel-news.com/ (дата обращения: 15.03.2026).

14. Stack Overflow. Developer community Q&A. — URL: https://stackoverflow.com/ (дата обращения: 15.03.2026).

15. Git SCM. Official Git documentation. — URL: https://git-scm.com/doc (дата обращения: 15.03.2026).

16. Composer. Dependency Manager for PHP. — URL: https://getcomposer.org/doc/ (дата обращения: 15.03.2026).

17. Regard.ru — Онлайн-конфигуратор ПК. — URL: https://www.regard.ru/configurator (дата обращения: 15.03.2026).

18. DNS-shop.ru — Онлайн-конфигуратор ПК. — URL: https://www.dns-shop.ru/configurator/ (дата обращения: 15.03.2026).

19. Citilink.ru — Онлайн-конфигуратор ПК. — URL: https://www.citilink.ru/configurator/ (дата обращения: 15.03.2026).

20. Силбершатц А., Корф Г., Сударшан С. Базы данных. Проектирование, реализация и сопровождение. — 6-е изд. — М.: Вильямс, 2022. — 816 с.

21. Немчин А. Laravel для начинающих. — М.: Эксмо, 2024. — 320 с.

22. Google Developers. Web Fundamentals. — URL: https://developers.google.com/web/fundamentals (дата обращения: 15.03.2026).

23. PCI-SIG. PCI Express® Base Specification. — URL: https://pcisig.com/specifications (дата обращения: 15.03.2026).

24. Intel®. Intel ARK — Product Specifications. — URL: https://ark.intel.com/ (дата обращения: 15.03.2026).

25. Tom's Hardware. GPU hierarchy / GPU reviews. — URL: https://www.tomshardware.com/reviews/gpu-hierarchy,4388.html (дата обращения: 15.03.2026).

Приложение А

Таблицы(миграции):

Миграция users:

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('first_name', 64);
    $table->string('last_name', 64);
    $table->string('email', 254)->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->string('role', 30)->default('user');
    $table->rememberToken();
    $table->timestamps();
});
```

Миграция categories:

```
Schema::create('categories', function (Blueprint $table) {
    $table->string('category_id', 32)->primary();
    $table->string('name', 128);
    $table->timestamps();
});
```

Миграция products:

```
Schema::create('products', function (Blueprint $table) {
    $table->string('product_id', 64)->primary();
    $table->string('category_id', 32);
$table->foreign('category_id')->references('category_id')->on('categories')->onDelete('cascade');
    $table->string('sku', 100)->nullable();
```

```
        $table->string('name', 255);
        $table->decimal('price', 10, 2)->default(0);
        $table->integer('stock_quantity')->default(0);
        $table->text('description')->nullable();
        $table->string('main_image_path', 255)->nullable();
        $table->boolean('is_active')->default(true);
        $table->timestamps();
    });
```

Миграция carts:

```
    Schema::create('cpus', function (Blueprint $table) {
        $table->string('product_id', 64)->primary();
        $table->foreign('product_id')
->references('product_id')->on('products')->onDelete('cascade');
        $table->string('socket', 32)->nullable();
        $table->integer('tdp_w')->nullable();
        $table->integer('max_mem_speed')->nullable();
        $table->string('brand', 64)->nullable();
        $table->integer('cores')->nullable();
        $table->integer('threads')->nullable();
        $table->decimal('base_clock', 4, 2)->nullable();
        $table->decimal('boost_clock', 4, 2)->nullable();
        $table->boolean('integrated_graphics')->default(false);
    });
```

Миграция motherboards:

```
    Schema::create('motherboards', function (Blueprint $table) {
```

```
            $table->string('product_id', 64)->primary();

$table->foreign('product_id')->references('product_id')->on('products')->onDelete('cascade');
            $table->string('socket', 32)->nullable();
            $table->string('form_factor', 32)->nullable();
            $table->integer('ram_slots')->default(2);
            $table->integer('max_ram')->nullable();
            $table->integer('ram_speed_max')->nullable();
            $table->integer('m2_slots')->default(0);
            $table->integer('pcie_version')->nullable();
            $table->integer('cpu_fan_headers')->default(1);
            $table->integer('sata_ports')->default(0);
            $table->string('chipset', 64)->nullable();
        });
```

Миграция rams:

```
        Schema::create('rams', function (Blueprint $table) {
            $table->string('product_id', 64)->primary();
            $table->foreign('product_id')
->references('product_id')->on('products')->onDelete('cascade');
            $table->string('ram_type', 8)->nullable();
            $table->integer('size_gb')->nullable();
            $table->integer('speed_mhz')->nullable();
        });
```

Миграция published_builds:

```
        Schema::create('published_builds', function (Blueprint $table) {
            $table->id();
```

```
        $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
        $table->string('name', 255);
        $table->text('description')->nullable();
        $table->json('build_data');
        $table->decimal('total_price', 10, 2)->default(0);
        $table->timestamps();
    });
```

Миграция build_votes:

```
    Schema::create('build_votes', function (Blueprint $table) {
        $table->id();
$table->foreignId('published_build_id')->constrained('published_builds')->onDelete('cascade');
        $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
        $table->tinyInteger('vote');
        $table->timestamps();
        $table->unique(['published_build_id', 'user_id']);
    });
```

Миграция build_comments:

```
    Schema::create('build_comments', function (Blueprint $table) {
        $table->id();
$table->foreignId('published_build_id')->constrained('published_builds')->onDelete('cascade');
        $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
        $table->text('text');
        $table->timestamps();
    });
```

Продолжение приложение А

Модели:

Модель User:

```
protected $fillable = [
    'first_name', 'last_name', 'email', 'password', 'role',
];

protected $hidden = ['password', 'remember_token'];

protected function casts(): array {
    return ['email_verified_at' => 'datetime', 'password' => 'hashed'];
}

public function isAdmin(): bool {
    return $this->role === 'admin';
}

public function isPrivileged(): bool {
    return in_array($this->role, ['admin', 'content_manager', 'warehouse_manager']);
}
```

Модель Product:

```
protected $primaryKey = 'product_id';
public $incrementing = false;
protected $keyType = 'string';

protected $fillable = [
```

```
        'product_id', 'category_id', 'sku', 'name', 'price',
        'stock_quantity', 'description', 'main_image_path', 'is_active',
    ];

    public function category() { return $this->belongsTo(Category::class, 'category_id', 'category_id'); }
    public function cpu() { return $this->hasOne(Cpu::class, 'product_id', 'product_id'); }
    public function motherboard() { return $this->hasOne(Motherboard::class, 'product_id', 'product_id'); }
    public function ram() { return $this->hasOne(Ram::class, 'product_id', 'product_id'); }
    public function gpu() { return $this->hasOne(Gpu::class, 'product_id', 'product_id'); }
    public function storage() { return $this->hasOne(Storage::class, 'product_id', 'product_id'); }
    public function psu() { return $this->hasOne(Psu::class, 'product_id', 'product_id'); }
    public function cooler() { return $this->hasOne(Cooler::class, 'product_id', 'product_id'); }
    public function pcCase() { return $this->hasOne(PcCase::class, 'product_id', 'product_id'); }
```

Модель PublishedBuild:

```
    protected $fillable = ['user_id', 'name', 'description', 'build_data', 'total_price'];

    protected $casts = ['build_data' => 'array'];

    public function user() { return $this->belongsTo(User::class); }
    public function votes() { return $this->hasMany(BuildVote::class, 'published_build_id'); }
    public function comments() { return $this->hasMany(BuildComment::class, 'published_build_id'); }
```

Модель Template:

```
    protected $primaryKey = 'template_id';
```

```
public $incrementing = false;
protected $keyType = 'string';

protected $fillable = ['template_id', 'name', 'author_user_id', 'is_public'];

public function items() { return $this->hasMany(TemplateItem::class, 'template_id', 'template_id'); }
public function author() { return $this->belongsTo(User::class, 'author_user_id'); }
```

Контроллеры:

Контроллер AuthController:

```
public function register(Request $request)
{
    $validator = Validator::make($request->all(), [
        'first_name' => 'required|string|max:64',
        'last_name' => 'required|string|max:64',
        'email' => 'required|email|max:254|unique:users,email',
        'password' => ['required', 'string', 'min:8', 'confirmed',
            Password::min(8)->letters()->mixedCase()->numbers()],
    ]);
    $validated = $validator->validateWithBag('register');

    $user = User::create([
        'first_name' => $validated['first_name'],
        'last_name' => $validated['last_name'],
        'email' => $validated['email'],
        'password' => $validated['password'],
```

```
            'role' => 'user',
        ]);

        Auth::login($user);
        $request->session()->regenerate();
        return redirect('/');
    }

    public function login(Request $request)
    {
        $credentials = $request->validate([
            'email' => 'required|email|max:254',
            'password' => 'required|string',
        ]);

        if (Auth::attempt($credentials, $request->boolean('remember'))) {
            $request->session()->regenerate();
            return redirect()->intended('/');
        }
        return back()->withErrors(['email' => 'Неверный email или пароль.'])->onlyInput('email');
    }

    public function logout(Request $request)
    {
        Auth::logout();
        $request->session()->invalidate();
```

```
        $request->session()->regenerateToken();
        return redirect('/login');
    }
```

Контроллер CatalogController:

```
    public function users(Request $request)
    {
        $query = User::query();
        if ($search = $request->input('search')) {
            $query->where('email', 'like', '%' . $search . '%');
        }
        return response()->json($query->orderBy('id')->get());
    }

    public function updateRole(Request $request, int $id)
    {
        $request->validate([
            'role' => 'required|string|in:user,content_manager,warehouse_manager,admin',
        ]);
        $user = User::findOrFail($id);
        $user->update(['role' => $request->role]);
        return response()->json(['success' => true, 'user' => $user]);
    }

    public function destroyUser(int $id)
    {
        $user = User::findOrFail($id);
```

```
        if ($user->id === Auth::id()) {
            Auth::logout();
        }
        $user->delete();
        return response()->json(['success' => true]);
    }
```

Маршруты (routes/web.php):

```
// Главная — конфигуратор
Route::get('/', [ConfiguratorController::class, 'index'])->name('configurator');

// Каталог сборок
Route::get('/builds', [BuildCatalogController::class, 'index'])->name('builds.index');
Route::get('/builds/{id}', [BuildCatalogController::class, 'show'])->name('builds.show');
Route::get('/api/builds', [BuildCatalogController::class, 'apiIndex']);

// Аутентификация
Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
Route::post('/login', [AuthController::class, 'login']);
Route::post('/register', [AuthController::class, 'register']);
Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

// API — публичные
Route::get('/api/user', [AuthController::class, 'user']);
Route::get('/api/products/{category}', [ProductController::class, 'byCategory']);
Route::get('/api/products/{category}/filters', [ProductController::class, 'filters']);
Route::get('/api/product/{productId}', [ProductController::class, 'show']);
```

Продолжение приложение А

```
Route::get('/api/templates', [BuildController::class, 'templates']);


// API — авторизованные пользователи
Route::middleware('auth')->group(function () {
    Route::post('/api/build/add-to-cart', [BuildController::class, 'addToCart']);
    Route::post('/api/build/export', [BuildController::class, 'exportBuild']);
    Route::post('/api/templates', [BuildController::class, 'storeTemplate']);
  Route::delete('/api/templates/{templateId}', [BuildController::class, 'destroyTemplate']);
    Route::post('/api/builds', [BuildCatalogController::class, 'store']);
    Route::post('/api/builds/{id}/vote', [BuildCatalogController::class, 'vote']);
    Route::post('/api/builds/{id}/comment', [BuildCatalogController::class, 'comment']);
});


// API — привилегированные пользователи
Route::middleware(['auth', PrivilegedMiddleware::class])->group(function () {
    Route::post('/api/products', [ProductController::class, 'store']);
    Route::put('/api/product/{productId}', [ProductController::class, 'update']);
    Route::delete('/api/product/{productId}', [ProductController::class, 'destroy']);
    Route::post('/api/product/upload-image', [ProductController::class, 'uploadImage']);
    Route::put('/api/templates/{templateId}', [BuildController::class, 'updateTemplate']);
    Route::delete('/api/builds/{id}', [BuildCatalogController::class, 'destroy']);
    Route::delete('/api/builds/comments/{id}', [BuildCatalogController::class, 'destroyComment']);
});


// Панель администратора
Route::middleware(['auth', AdminMiddleware::class])->group(function () {
```

Продолжение приложение А

```
    Route::get('/admin', [AdminController::class, 'index'])->name('admin');

    Route::get('/api/admin/users', [AdminController::class, 'users']);

    Route::put('/api/admin/users/{id}/role', [AdminController::class, 'updateRole']);

    Route::delete('/api/admin/users/{id}', [AdminController::class, 'destroyUser']);

});
```

Приложение В

Страница авторизации и регистрации

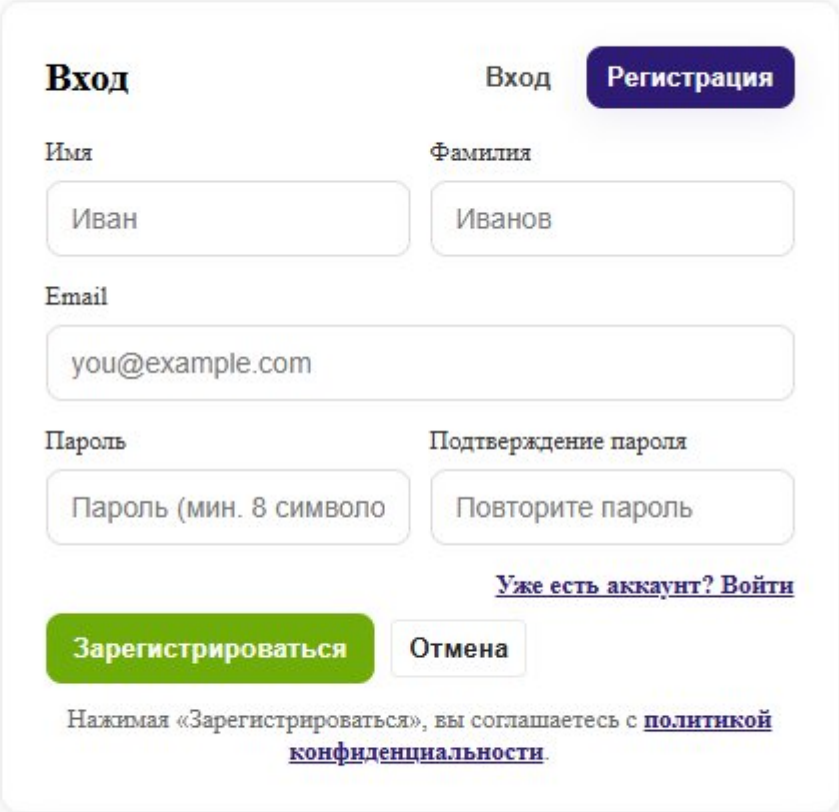

Рисунок 1 – Страница регистрации – форма регистрации

Продолжение приложение В

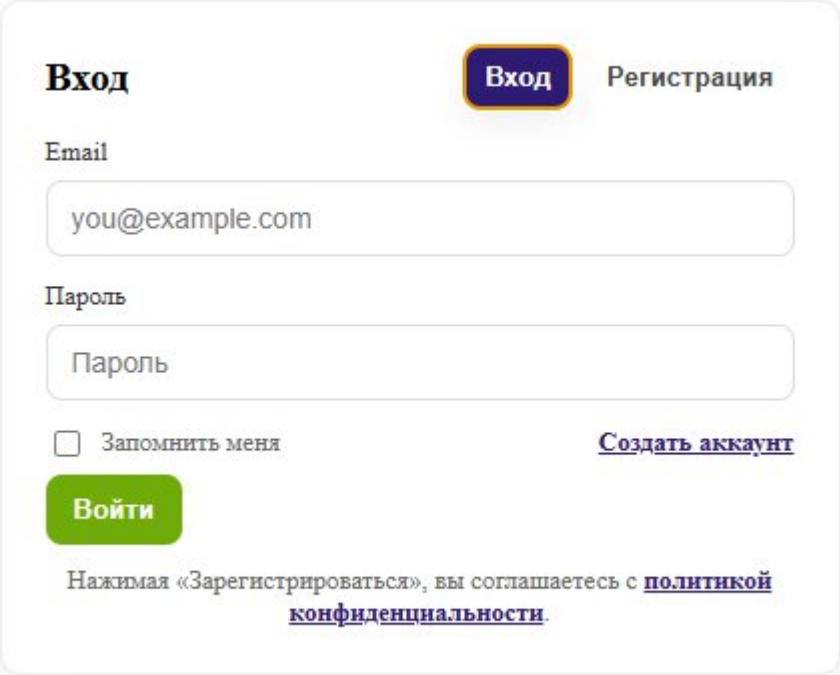

Рисунок 2 – Страница авторизации – форма авторизации

Продолжение приложение В

Страница конфигуратора

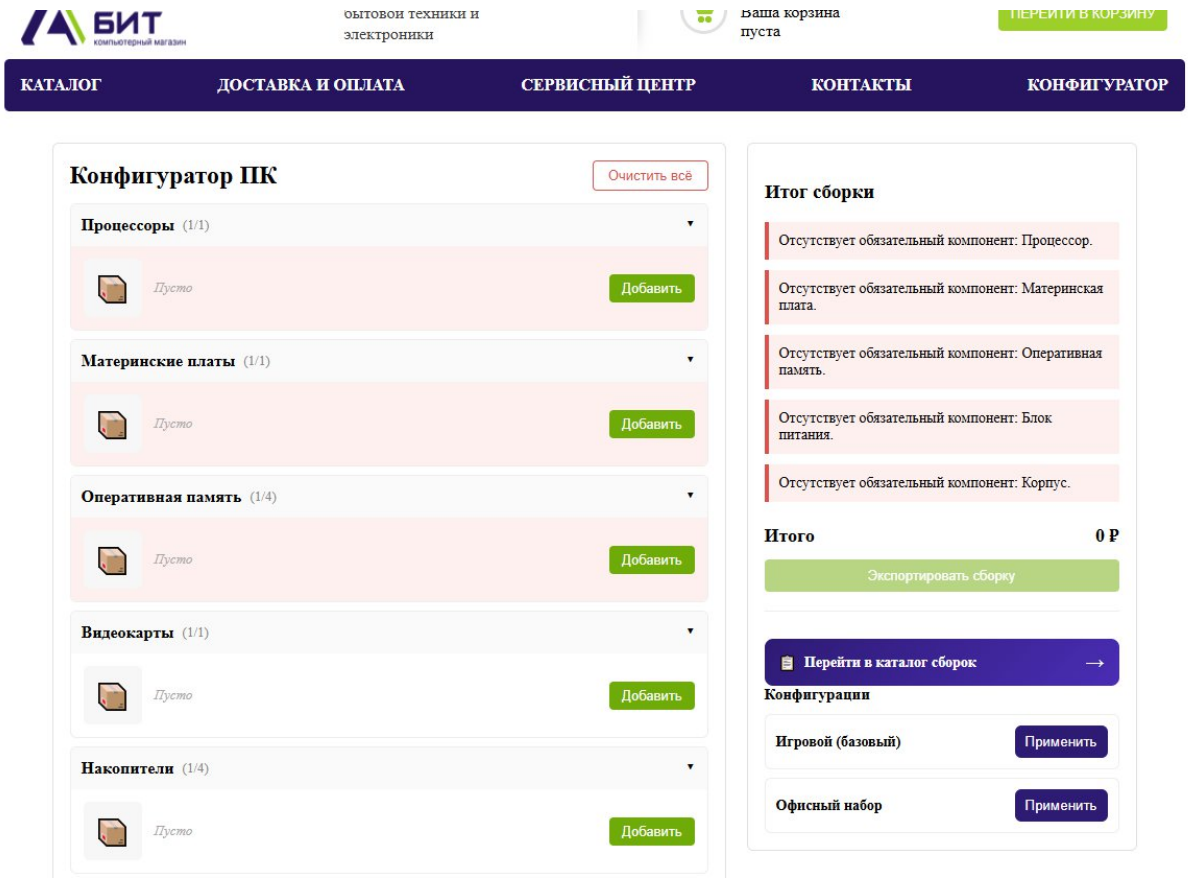

Рисунок 3 – Основная рабочая область конфигуратора

Продолжение приложение В

Модальное окно каталога компонентов

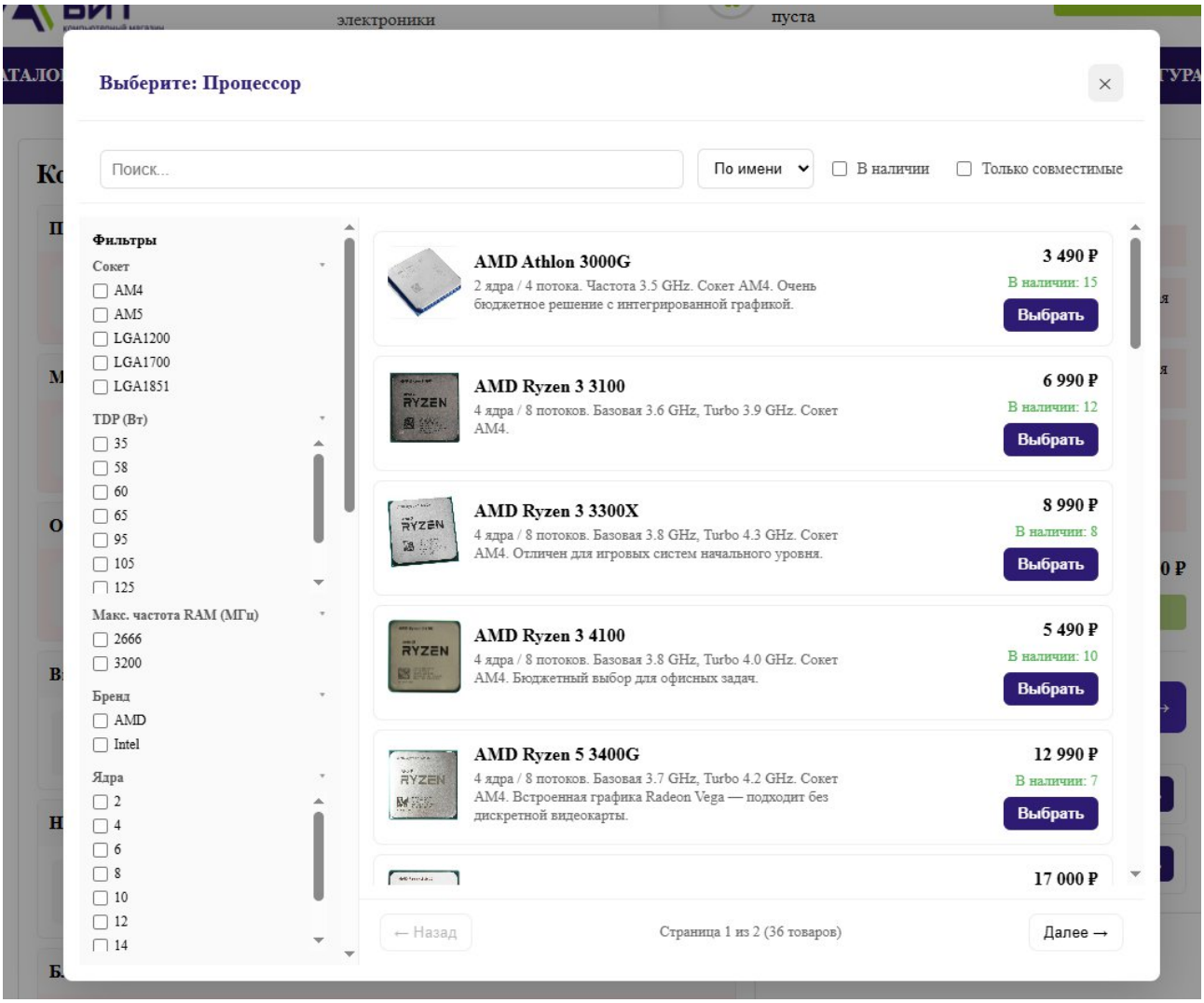

Рисунок 4 – Каталог компонентов с фильтрами

Продолжение приложение В

Рисунок 5 – Карточка товара

Продолжение приложение В

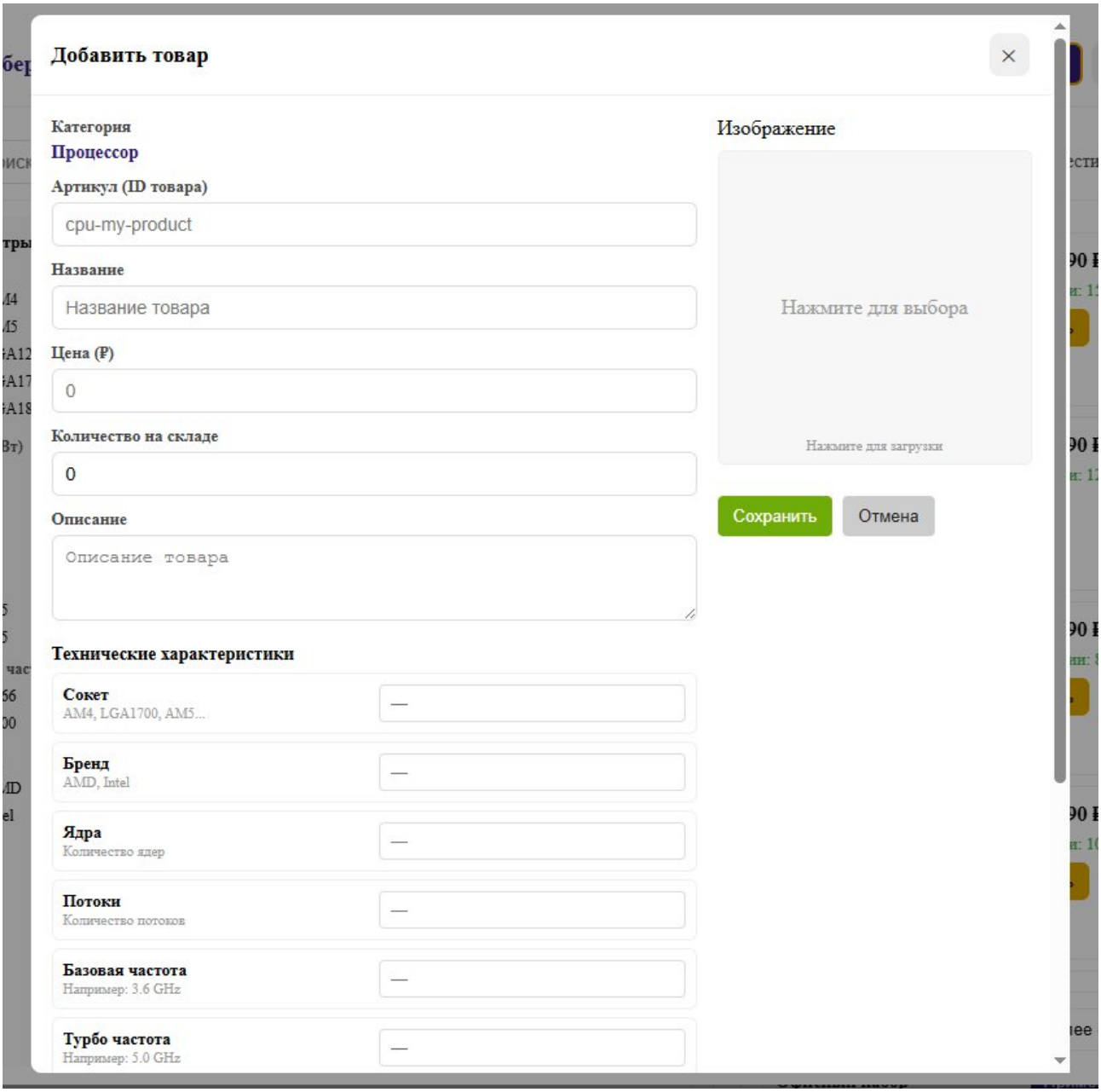

Рисунок 6 – Форма добавления товара (админ)

Продолжение приложение В

Шаблоны и публикация

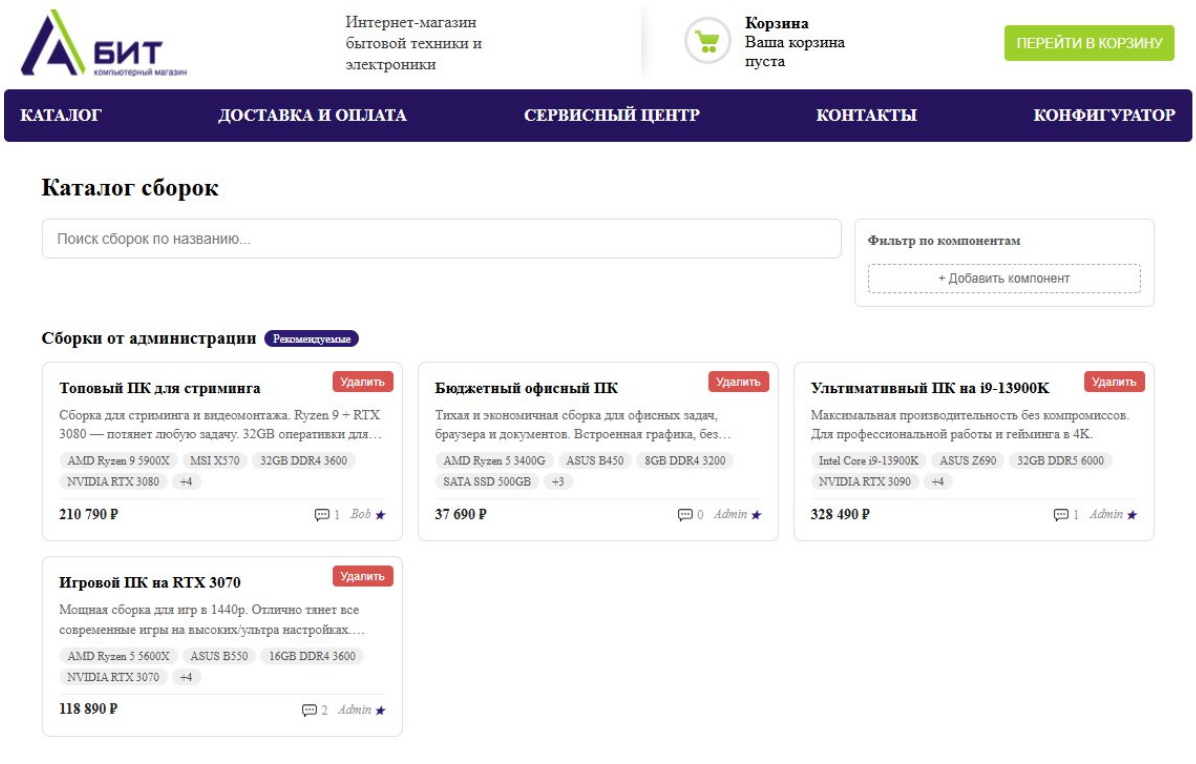

Рисунок 7 – Панель шаблонов

Продолжение приложение В

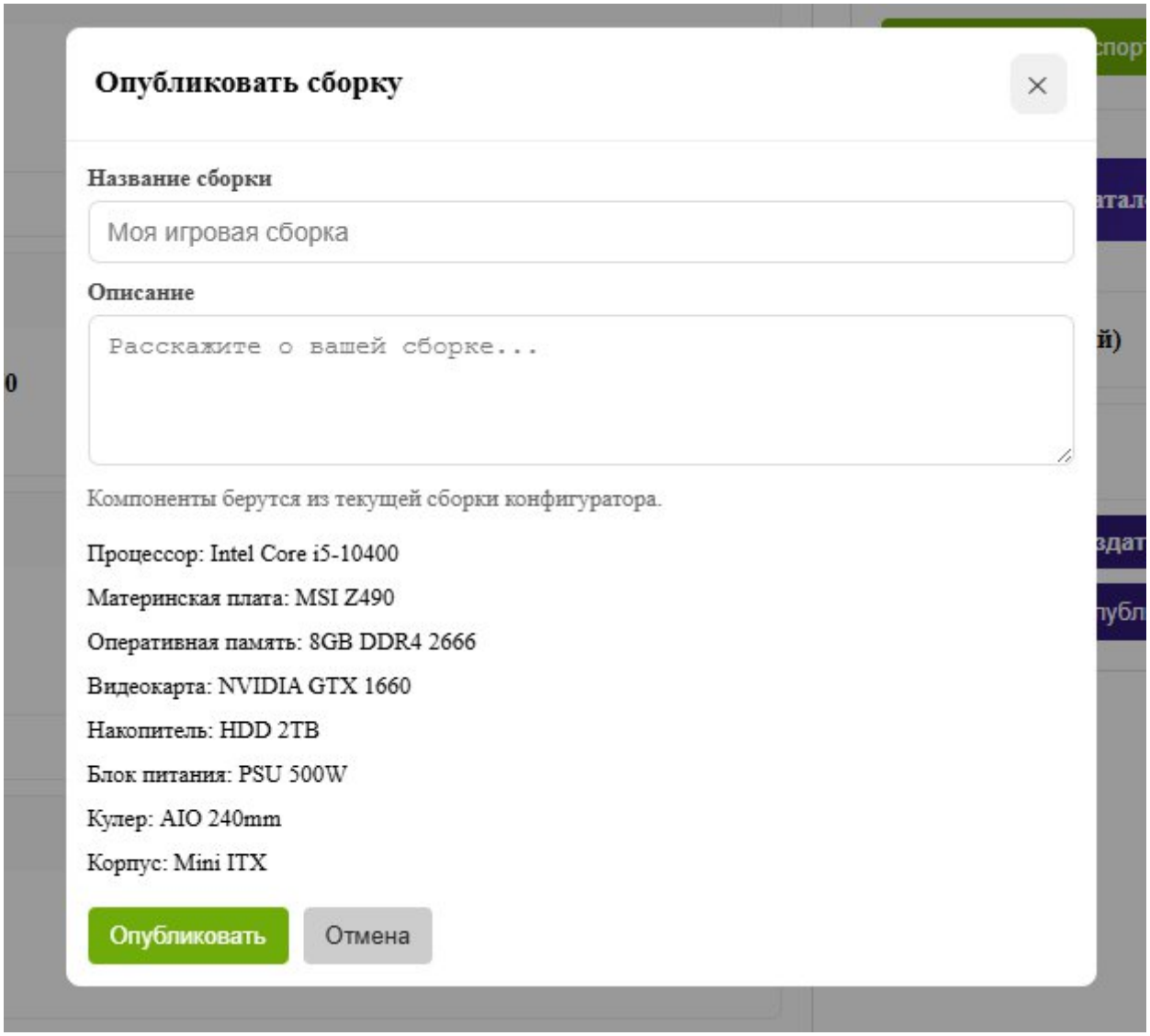

Рисунок 8 – Модальное окно публикации сборки

Каталог опубликованных сборок

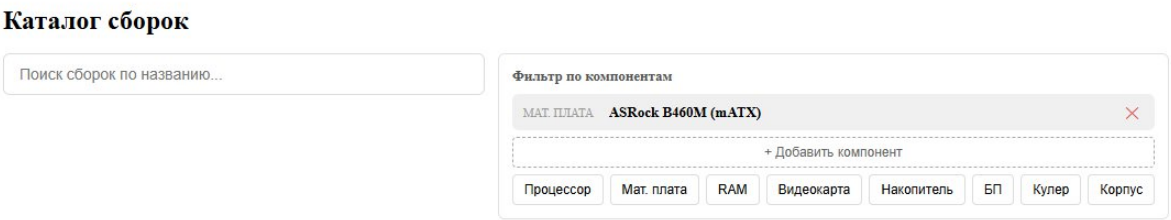

Рисунок 9 – Каталог сборок с поиском и фильтрацией

Продолжение приложение В

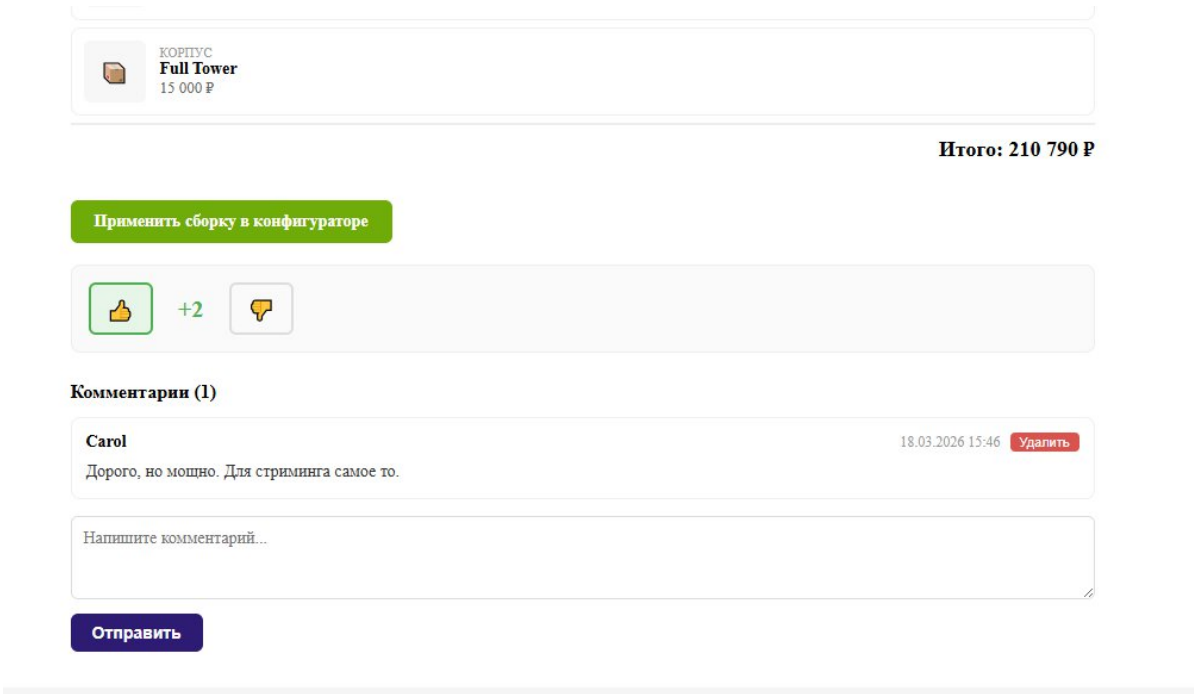

Рисунок 10 – Детальная страница сборки с голосованием и комментариями

Административная панель

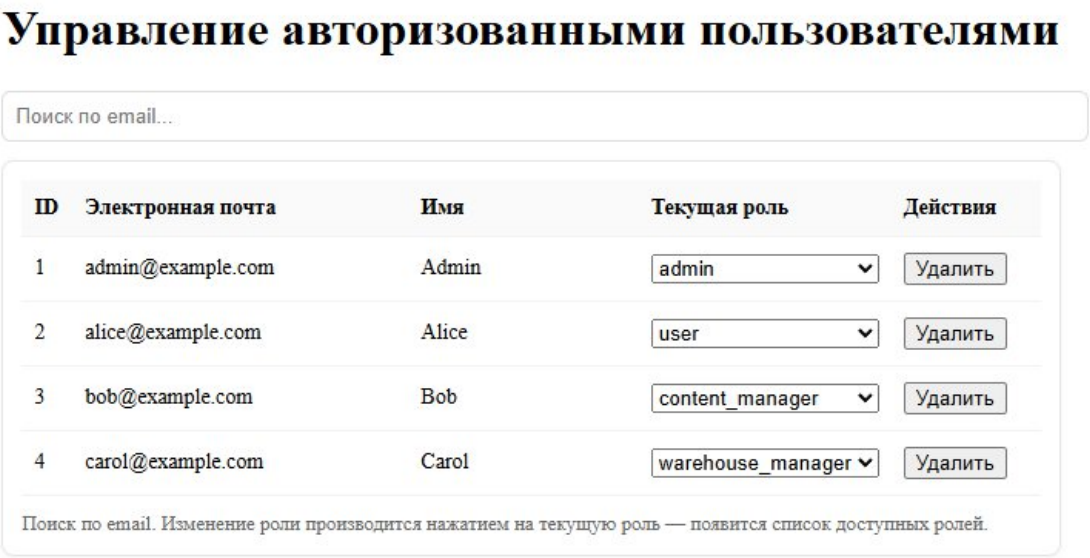

Рисунок 11 – Управление пользователями